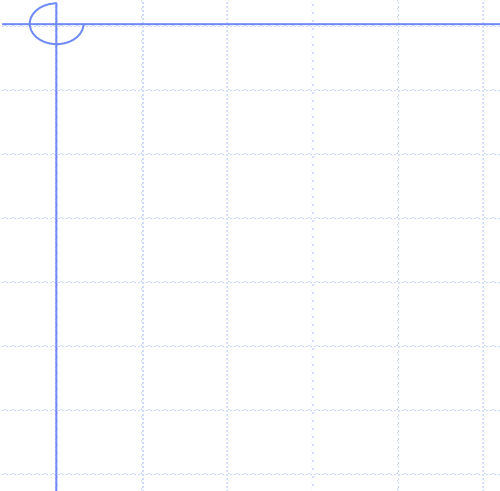
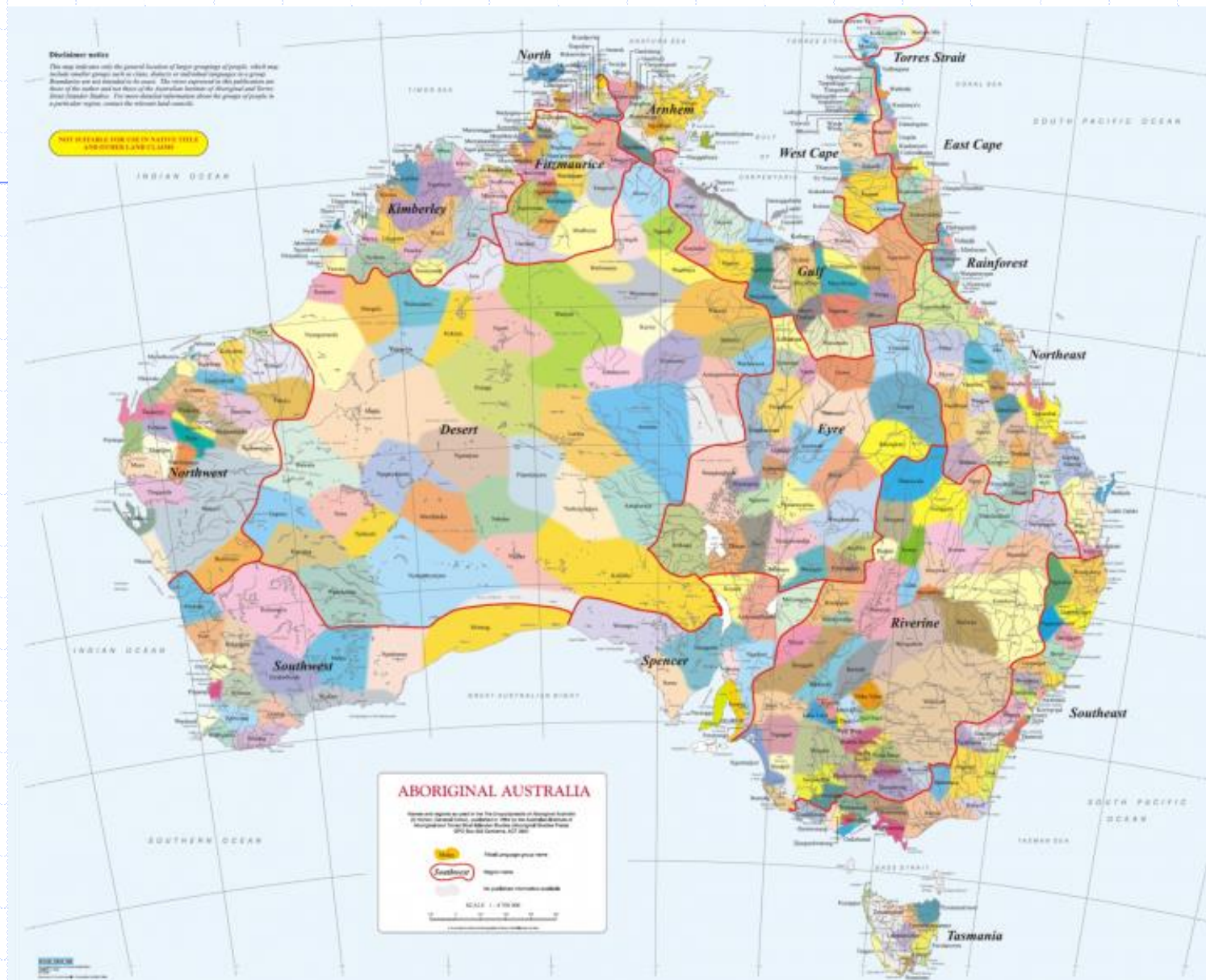




Week 1

Course Introduction

Algorithms and Data Structures
COMP3506/7505



Week 1 – Course Introduction

1. Boring but important admin stuff
2. Why are we here?
3. Introduction to data structures
4. Measuring algorithms – first take
5. Model of computation
6. Algorithm Analysis

Who are we?

- Dr Joel Mackenzie – Coordinator and Lecturer
 - You can call me Joel
- Fantastic Tutors
 - Connor, Hugh, Jonathan, Katya, Minhao, Mitchell, Ryan, Vladimir, Yutong, Zachary

Who are you?

- Extremely smart, well motivated, timely, ...
- About 600 students in total
 - Majority are COMP3506 (undergraduate)
 - About 10% typically COMP7505 (postgraduate)

Prerequisite Knowledge

- High School Maths
 - Basic algebra
 - Logarithms
 - Sequences and series
- Discrete Mathematics (eg. MATH1061/7861)
 - Graphs
 - Set Theory and Relations
 - Basic Proofs
- Programming Skills
(CSSE1001/7030 & CSSE2002/7023)
 - Object-oriented programming
 - The Java/**Python** programming language

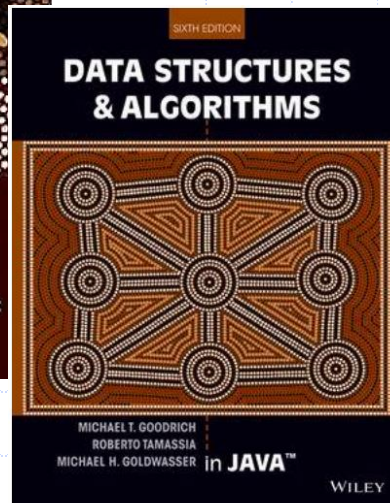
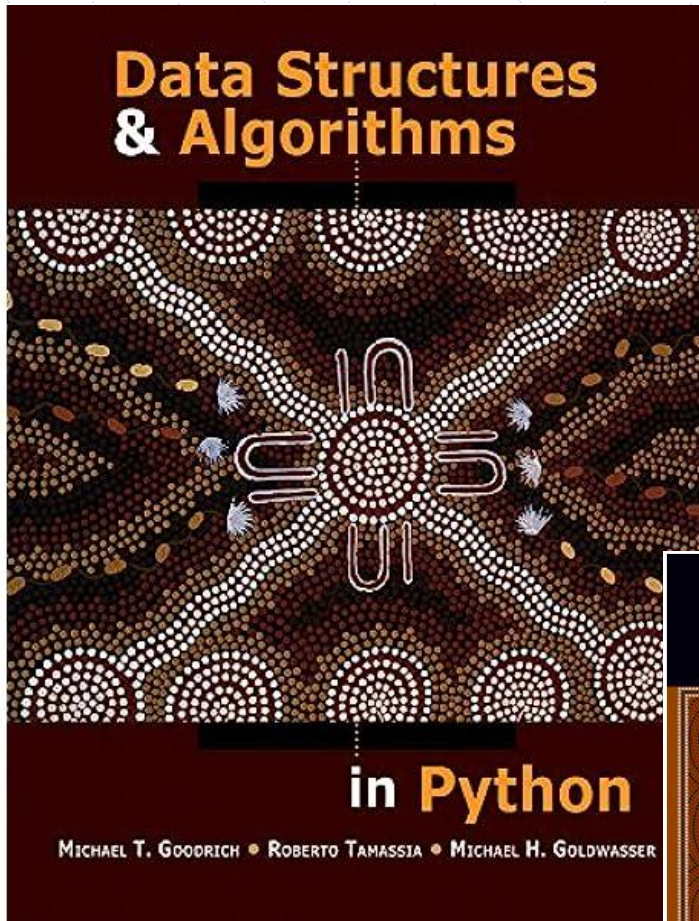
Data Structures and Algorithms in *Python??*

- This is **not** a programming course
 - Techniques we explore in this course are *language agnostic*
- The goal of this course is to make you **think like a computer scientist**
- I wanted to teach it in C but...

What you'll learn in this course

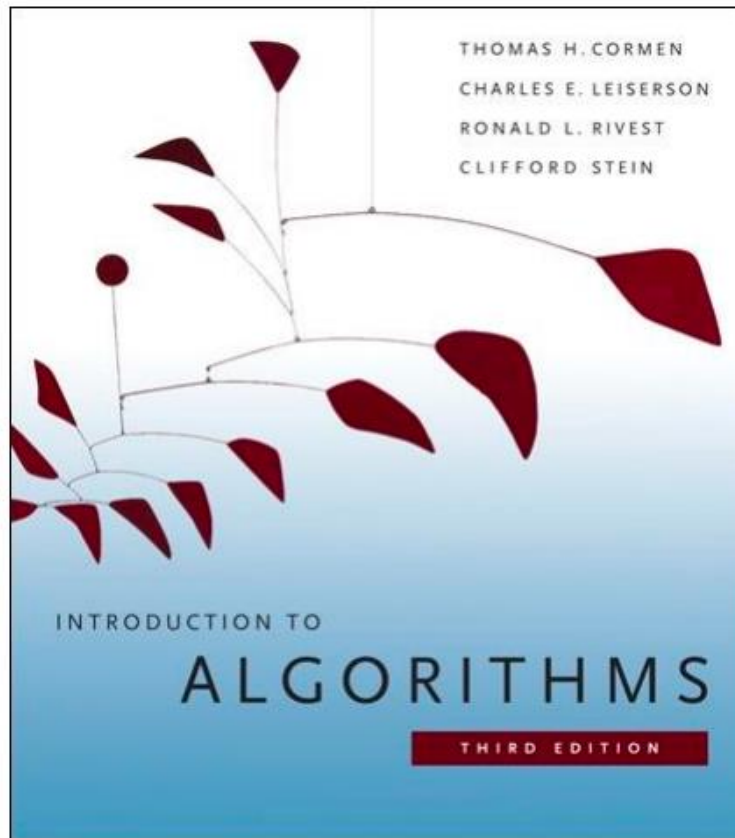
- ❑ The goal of this course is to build an understanding of **data structures** and their associated **algorithms** so that you can select an **appropriate** set when designing software
- ❑ These algorithms and data structures form the basis of **large and complex software systems** and are necessary in order to create **efficient** and **scalable** programs
- ❑ **Think like a computer scientist!**

Learning Resources



- ❑ *Data Structures and Algorithms in Python.*
M. T. Goodrich, R. Tamassia and M.H. Goldwasser
 - Also known as “GTG”
- ❑ Many lectures will correspond to a chapter in this book

Learning Resources



- *Introduction to Algorithms*. 3rd ed.
T.H. Cormen, C.E. Leiserson,
R.L. Rivest and C. Stein
 - Also known as “CLRS”
- Good, in-depth, coverage of algorithms and interacting with data structures.
- Extends the topics discussed in lectures and tutorials, and covers more advanced topics.

Learning Resources

- **Blackboard**

- Lecture slides, tutorial sheets, lecture recordings, assignment specifications

- **Ed Discussion**

- Does what it says on the tin

- **Gradescope**

- Quizzes and assignment submissions

Learning Activities

□ Lectures

- **2 hours per week**
- Thursdays from 12-2pm, 49-200
- Recorded via Echo360

□ Tutorials

- **2 hours per week**
- Start in week 2
- Very important!

Weekly drop in hour

- Appointment only
- 3-4pm Thursday
- GP-South: 78-546

Assessment

Assessment	COMP3506 Weighting	COMP7505 Weighting
Quizzes	10% (7 quizzes, 2% each)	10% (7 quizzes, 2% each)
Assignment 1	15%	15% (+ extra question)
Assignment 2	25%	25% (+ extra question)
Final exam – closed book	50%	50%

Question: How does “7 quizzes, 2% each” = 10%?

Quizzes released on Monday mornings. Due Friday afternoon at 3pm. Do the tutorial before the quiz!

Check the ECP for more information (including due dates)

Also check out the “Plan at a Glance” document

COMP3506/7505 Weekly Plan: Version 1 – July 4, 2023

Week	Lecture	Tutorial	Readings	Assessment Due
1	Intro, computation models, basic algorithm analysis, asymptotic analysis	–	GTG: 3.1, 4 CLRS: 2.1, 2.2	–
2	Recursion, analysis of algorithms, comparison-based sorting	Algorithm analysis I	GTG: 12.1–12.3 CLRS: 2.3, 3, 4, 7	Quiz 1
3	Linear sorting, Lists and arrays, amortization, linked data structures	Algorithm analysis II	GTG: 6, 7.1–7.3, 12.4 CLRS: 8, 10.1–10.2	Quiz 2
4	Stacks, queues, iterators, trees	Sorting	GTG: 1.8, 5, 8 CLRS: 10.4	Quiz 3
5	Priority queues, heaps, adaptable priority queues, maps, sets, multimaps	Stacks, queues, trees	GTG: 9, 10.1, 10.5 CLRS: 6	–
6	Hashing and hash tables, binary search trees	Priority queues and heaps	GTG: 10.2–10.3, 11.1–11.3 CLRS: 11, 12	Assignment 1 (15%)
7	Balanced Trees (AVL, Splay, 2-4, red-black, ...)	Maps and hash tables	GTG: 11.3–11.6 CLRS: 13, 18	Quiz 4
8	Graph basics, graph traversal, directed graph algorithms	Search trees	GTG: 14.1–14.5 CLRS: 22	Quiz 5
9	Shortest path algorithms, minimum spanning trees	Graph Theory I	GTG: 14.5–14.7 CLRS: 21, 23, 24	Quiz 6
<i>Mid Semester Break</i>				
10	Strings, pattern matching, tries	Graph Theory II	GTG: 13.1–13.2, 13.4–13.5 CLRS: 16.3, 32	–
11	Huffman Encoding, Text Compression	String Algorithms	GTG: 10.4, 12.7 CLRS: 9	Assignment 2 (25%)
12	Advanced applications and practical algorithms	Exam Revision I	–	Quiz 7
13	Course review	Exam Revision II	–	–
Exam Period	–	–	–	Exam (50%)

Plagiarism

Plagiarism is the act of misrepresenting as one's own original work the ideas, interpretations, words or creative works of another. These include published and unpublished documents, designs, music, sounds, images, photographs, computer codes and ideas gained through working in a group. These ideas, interpretations, words or works may be found in print and/or electronic media.

Students are encouraged to read the UQ Academic Integrity and Plagiarism policy (<http://ppl.app.uq.edu.au/content/3.60.04-student-integrity-and-misconduct>) which makes a comprehensive statement about the University's approach to plagiarism, including the approved use of plagiarism detection software, the consequences of plagiarism and the principles associated with preventing plagiarism.

Plagiarism

- ❑ Please discuss problems and potential approaches
 - Arguing with your friends is fun!
- ❑ Please look things up online
 - Sometimes alternative perspectives are helpful
- ❑ Don't copy answers
- ❑ Don't blindly use ChatGPT
- ❑ Don't pay someone to do your assignment
- ❑ Planning to cheat? Come and seek help.

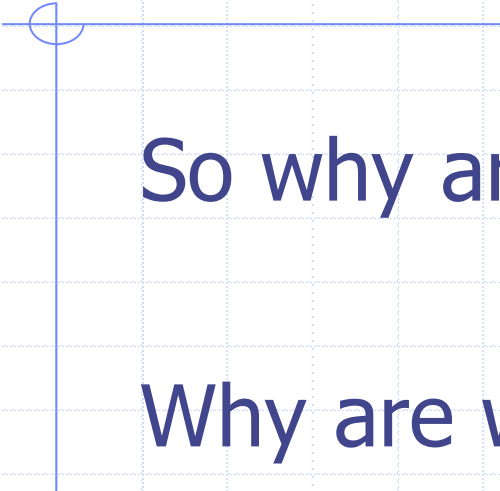
Some important policies...

- Extensions: Via my.uq – Please try to make these early.
 - No quiz extensions will be considered, ever, full stop.
- Late Policy (EECS wide)
 - 100% loss of marks with 1 hour grace period
 - Due at 3pm – Submit at 3.59pm, no worries. Submit at 4pm, you get a zero.

Week 1 – Course Introduction

1. Boring but important admin stuff
- 2. Why are we here?**
3. Introduction to data structures
4. Measuring algorithms – first take
5. Model of computation
6. Algorithm Analysis

Motivation and Introduction



So why are we here?

Why are we studying algorithms and data structures?

Algorithms are everywhere!

- ❑ Tie your shoelaces
- ❑ Making coffee
- ❑ Sorting your bookshelf
- ❑ Unlock your phone
- ❑ Traffic light sequences
- ❑ Recipes and cooking
- ❑ TikTok/Facebook/Whatever is cool now??
- ❑ And so on...

Data is exploding!!11!

- Yes, an old cliché...
- Computers are pretty fast today, so why do we care?
- Many problems we want to solve today have input data that is so large that we **do not have a choice**
 - If we cannot do things efficiently, we cannot do them at all

Web Search

A white search bar with a thin grey border and a small microphone icon on the right side.

Google Search

I'm Feeling Lucky

- ❑ Algorithms
 - Crawl the web (web crawlers)
 - Process web pages for keywords
 - Retrieve and rank relevant information for your queries
- ❑ Data structures
 - Cache web pages
 - Store keywords and page URLs
 - Indices of keywords
 - URLs and cached site content

What will you get out of today?

1. **Motivation** for the analysis of algorithms
2. An **exploration** of possible methods for comparing algorithms and their **shortcomings**
3. A basic model of **computation** and the related **mathematical tools** (analysis techniques) that we will use throughout the remainder of the course
4. You will start believing that maybe, just maybe, algorithms can be fun!

Week 1 – Course Introduction

1. Boring but important admin stuff
2. Why are we here?
- 3. Introduction to data structures**
4. Measuring algorithms – first take
5. Model of computation
6. Algorithm Analysis

Algorithms? Problems??

- Goal: Solve computational problems with algorithms
- A **problem** is a binary relation from **inputs** to **correct outputs**
- An **algorithm** is a **procedure** for mapping each **input** to some **output**

Solving a problem

- An algorithm *solves* a problem if:
 - For all possible inputs, it returns the **correct** output

That is, the algorithm is a sequence of computational steps that transform the input to an output.

- Often, there are too many inputs to check them all... we often need to reason about correctness
 - That can be done in terms of mathematical proofs
 - ◆ Induction, covered in MATH1061, revisited in COMP4500

Data Structures

- What is a data structure?
- Data structures define **how** we store and modify non-constant data
 - Allows us to support a set of operations on that data
 - We think of data structures in terms of *Abstract Data Types*
 - ◆ **Different data structures may implement the same interface with different performance**

ADTs vs Data Structures

- Abstract Data Types (ADTs, *interfaces, APIs*)
 - Specifies what data we can store
 - What operations we can support and what they mean
- Data structures
 - Tell us **how** to store the data
 - Tell us **how** to support those operations
 - ◆ These are algorithms!
- Data structures “implement” ADTs

Static Sequence ADT

- Given a list of items X in some order: $x_1, x_2, \dots, x_n$

`build(X)` : Make new data structure for items in X

`len(X)` : Return n

`get(i)` : Return the element at position i

`set(i, x)` : Set x_i to x

- Note that the way we store the data and compute those functions depend on the *data structure we use*

Week 1 – Course Introduction

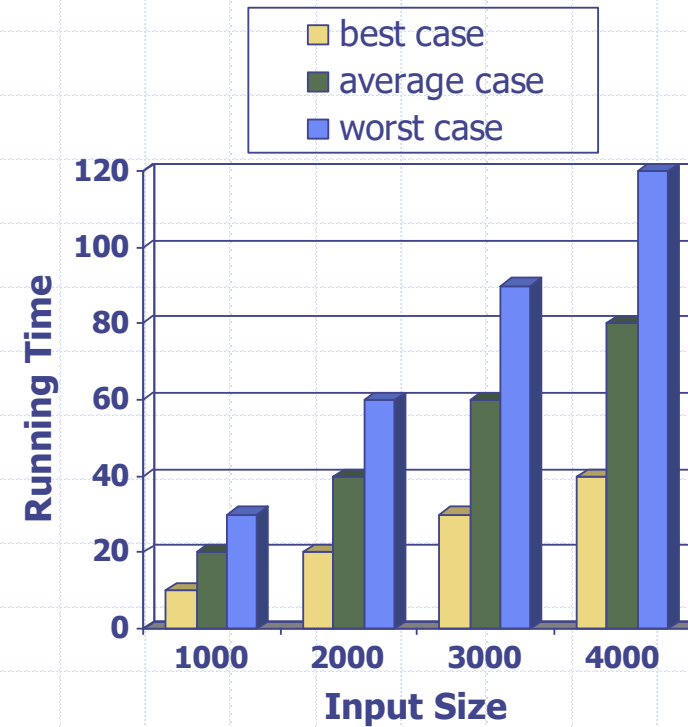
1. Boring but important admin stuff
2. Why are we here?
3. Introduction to data structures
- 4. Measuring algorithms – first take**
5. Model of computation
6. Algorithm Analysis

Algorithm Analysis

How does an algorithm's **runtime** and **memory** usage increase with the **size of the input**?

Running Time

- ❑ Algorithms transform input data into output data
- ❑ Running time of an algorithm typically grows with input size
- ❑ *Average* case time is often difficult to determine
- ❑ We focus on the *worst* case running time
 - easier to analyse
 - crucial for many real-world applications



Best, Average, Worst Cases

Given a list of unsorted numbers, L , and a specific number, k , Return true if k is in L , or false otherwise.

Consider the following algorithm to solve this problem:

```
def number_exists(L, k):  
    for item in L:  
        if item == k:  
            return True  
    return False
```

- What is the **best**, **average**, and **worst** case?

How Can We Analyse Algorithms?



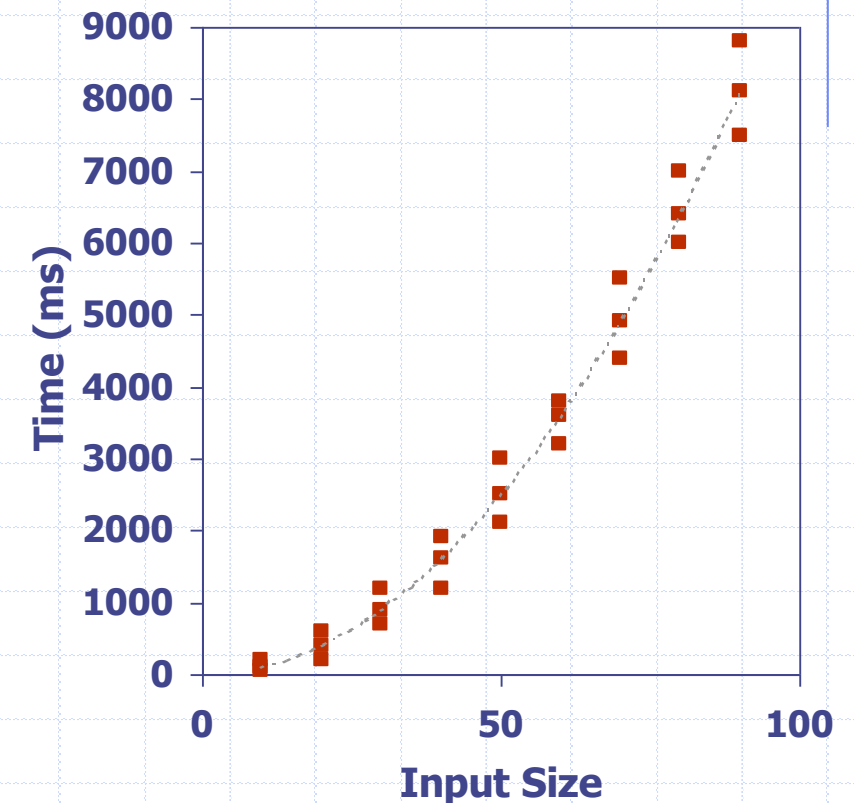
- Experimental (empirical) studies
- Theoretical analysis

Experimental/Empirical Studies

- ❑ Write program implementing the algorithm
- ❑ Run program with inputs of varying size and composition
- ❑ Use a method like `time.time()` to measure actual running time

```
from time import time
start_time = time( )
run algorithm
end_time = time( )
elapsed = end_time - start_time
```

- ❑ Plot the results



Limitations of Empirical Experiments ☹️

- ❑ Need to **implement** the algorithm
 - May be difficult, time consuming, ...
- ❑ Time may differ based on the **implementation**
 - Example: Finding the maximum element in a list
- ❑ Time may differ based on the **hardware**
- ❑ Results may not be indicative of the running time on **other inputs** not included in the experiment
 - How do I know if my inputs are best, average, or worst case?

Theoretical Analysis 😊

- Use a high-level description of the algorithm
 - instead of an implementation
- Characterise running time as a function of input size n
- Takes into account *all possible inputs*
 - at least those that are “bad” (hence, worst-case)
- Evaluation is independent of the hardware and software environment!

Theoretical Analysis Steps

1. **Express algorithm as pseudo-code**
2. Count primitive operations
3. Describe algorithm as $f(n)$
 - function of n (the input size)
4. Perform asymptotic analysis
 - express in **asymptotic notation**

Pseudo-Code Details

- Control flow
 - if ... then ... [else ...]
 - while ... do ...
 - repeat ... until ...
 - for ... do ...
 - Indentation replaces braces
- Method declaration

Algorithm *method* (*arg* [, *arg*...])
Input ...
Output ...
- Method call

method (*arg* [, *arg*...])
- Return value

return *expression*
- Mathematical formatting
 - n^2
- Expressions
 - $\leftarrow$ (*Assignment*)
 - ◆ Like "=" in Python
 - = (*Equality testing*)
 - ◆ Like "==" in Python

Find Max Element of an Array

Algorithm *arrayMax*(*A*, *n*)

Input array *A* of *n* integers

Output maximum element of *A*

currentMax $\leftarrow A[0]$

i $\leftarrow 1$

while *i* < *n* **do**

if *A*[*i*] > *currentMax* **then**

currentMax $\leftarrow A[i]$

i $\leftarrow i + 1$

return *currentMax*

Pseudo-code

```
def array_max(my_array):
```

```
    current_max = my_array[0]
```

```
    for element in my_array:
```

```
        if element > current_max:
```

```
            current_max = element
```

```
    return current_max
```

Python code

Different Styles... No worries – just be consistent

Algorithm 1: An algorithm with caption

Data: $n \geq 0$

Result: $y = x^n$

$y \leftarrow 1;$

$X \leftarrow x;$

$N \leftarrow n;$

while $N \neq 0$ **do**

if N is even **then**

$X \leftarrow X \times X;$

$N \leftarrow \frac{N}{2};$ /* This is a comment */

else

if N is odd **then**

$y \leftarrow y \times X;$

$N \leftarrow N - 1;$

end

end

end

Algorithm 1 An algorithm with caption

Require: $n \geq 0$

Ensure: $y = x^n$

$y \leftarrow 1$

$X \leftarrow x$

$N \leftarrow n$

while $N \neq 0$ **do**

if N is even **then**

$X \leftarrow X \times X$

$N \leftarrow \frac{N}{2}$

▷ This is a comment

else if N is odd **then**

$y \leftarrow y \times X$

$N \leftarrow N - 1$

end if

end while

Theoretical Analysis Steps

1. Express algorithm as pseudo-code
2. **Count primitive operations**
3. Describe algorithm as $f(n)$
 - function of n (the input size)
4. Perform asymptotic analysis
 - express in **asymptotic notation**

But what is an “operation”??

- In this course, we will assume the **Word-RAM** model
 - A word is a sequence of w bits (most of our computers have $w = 64$ bits)
 - Basic arithmetic operations take a **single operation**
 - ♦ $+$ $-$ $\%$ $*$ $//$ and so on
 - Bitwise operands take a **single operation**
 - ♦ $\&$, $|$, $<<$, $>>$, ...
 - Comparisons take a **single operation**
 - ♦ $>$, $<$, $==$, $!=$
 - Accessing or writing a word in memory takes a **single operation**
 - Most modern computers are *byte addressable*
 - ♦ Need to be able to access every memory “cell” limits memory size
 - ♦ $w \geq \# \text{bits required to represent the largest memory address}$

Counting Primitive Operations

Assignment

num = 10 1 operation

1. Assigning a value to a variable

num = *A*[10] 2 operations

1. Indexing into an array (memory lookup)
2. Assigning a value to a variable

Counting Primitive Operations

Control Flow

if num < 10: 1 operation

... add operations

if num < A[10]: 2 operations

... add operations

Counting Primitive Operations

Loops

`while i < 10:` 1 operation per iteration + 1

... counted operations times loop

Counting Primitive Operations

Loops

[Yes, Python handles for loops a bit differently]

```
for (int i=0; i < n; i++)
```

$1 + (n + 1)$

```
{
```

```
    ... let counted operations =  $X$ 
```

$n \cdot X$

```
} // count increment
```

n

- "`int i=0`" is executed once
- "`i < n`" conditional is evaluated $n + 1$ times
- "`i++`" increment is evaluated n times
- X instructions within loop are evaluated n times

Theoretical Analysis Steps

1. Express algorithm as pseudo-code
2. Count primitive operations
3. **Describe algorithm as $f(n)$**
 - function of n (the input size)
4. Perform asymptotic analysis
 - express in **asymptotic notation**

Describe as $f(n)$ – Function of n

- By inspecting the pseudo-code, we can determine the **maximum** number of primitive operations executed by an algorithm, as a function of the input size

Algorithm <i>arrayMax</i> (<i>A</i> , <i>n</i>)	<u># operations</u>
<i>currentMax</i> $\leftarrow$ <i>A</i> [0]	2
<i>i</i> $\leftarrow$ 1	1
while <i>i</i> < <i>n</i> do	<i>n</i>
if <i>A</i> [<i>i</i>] > <i>currentMax</i> then	$2 \cdot (n - 1)$
<i>currentMax</i> $\leftarrow$ <i>A</i> [<i>i</i>]	$2 \cdot (n - 1)$
<i>i</i> $\leftarrow$ <i>i</i> + 1	$n - 1$
return <i>currentMax</i>	1
Total:	$6 \cdot n + \text{lower order terms}$

Estimating Running Time

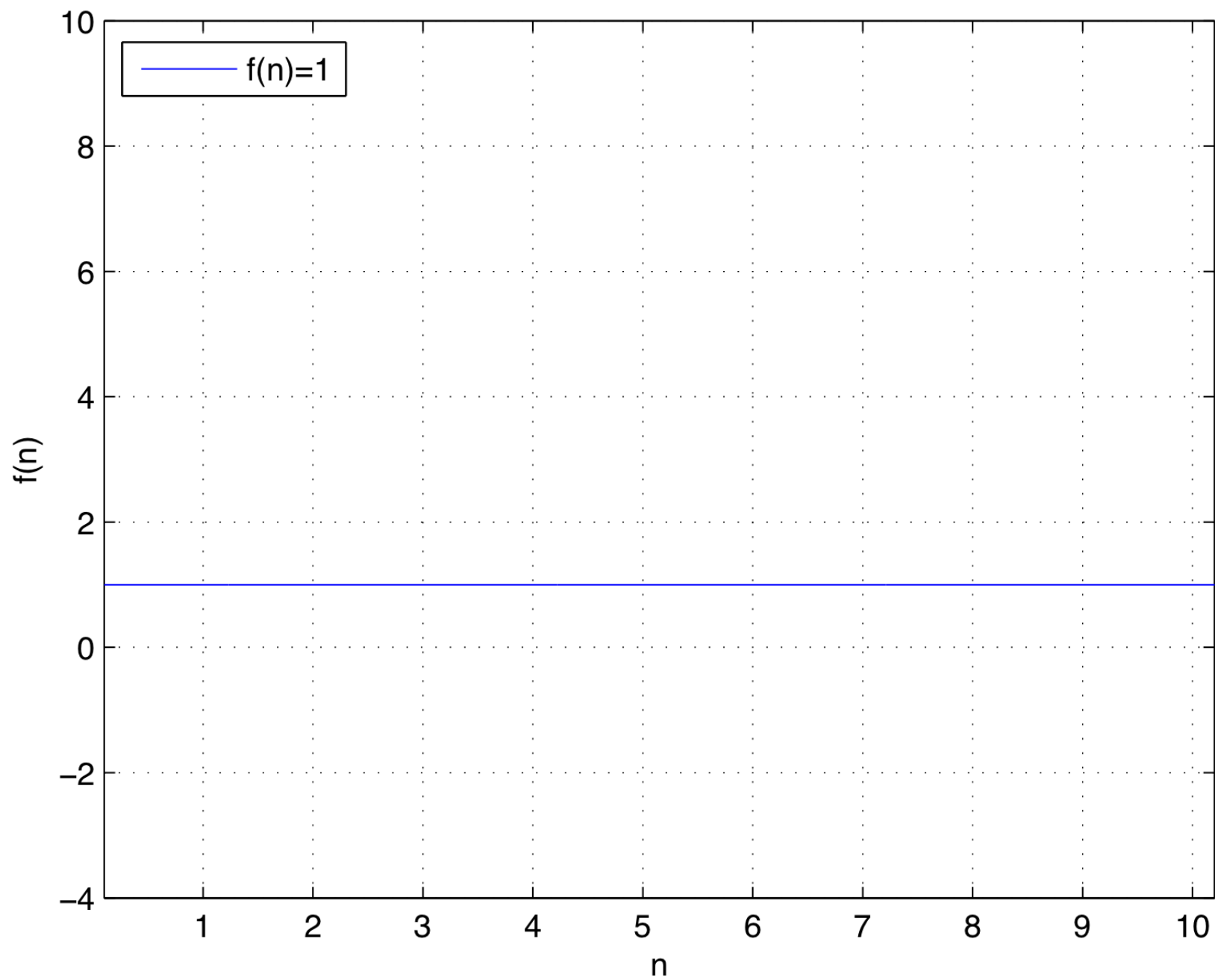
- *arrayMax* executes $6n$ primitive operations in the worst case
 - a = time taken by the fastest primitive operation
 - b = time taken by the slowest primitive operation
- Let $T(n)$ be worst-case time of *arrayMax*
 - $a \cdot 6n \leq T(n) \leq b \cdot 6n$
- Run time $T(n)$, is bounded by two **linear** functions
 - This tells us something VERY useful about our worst-case behaviour. Let's into this idea in more detail...

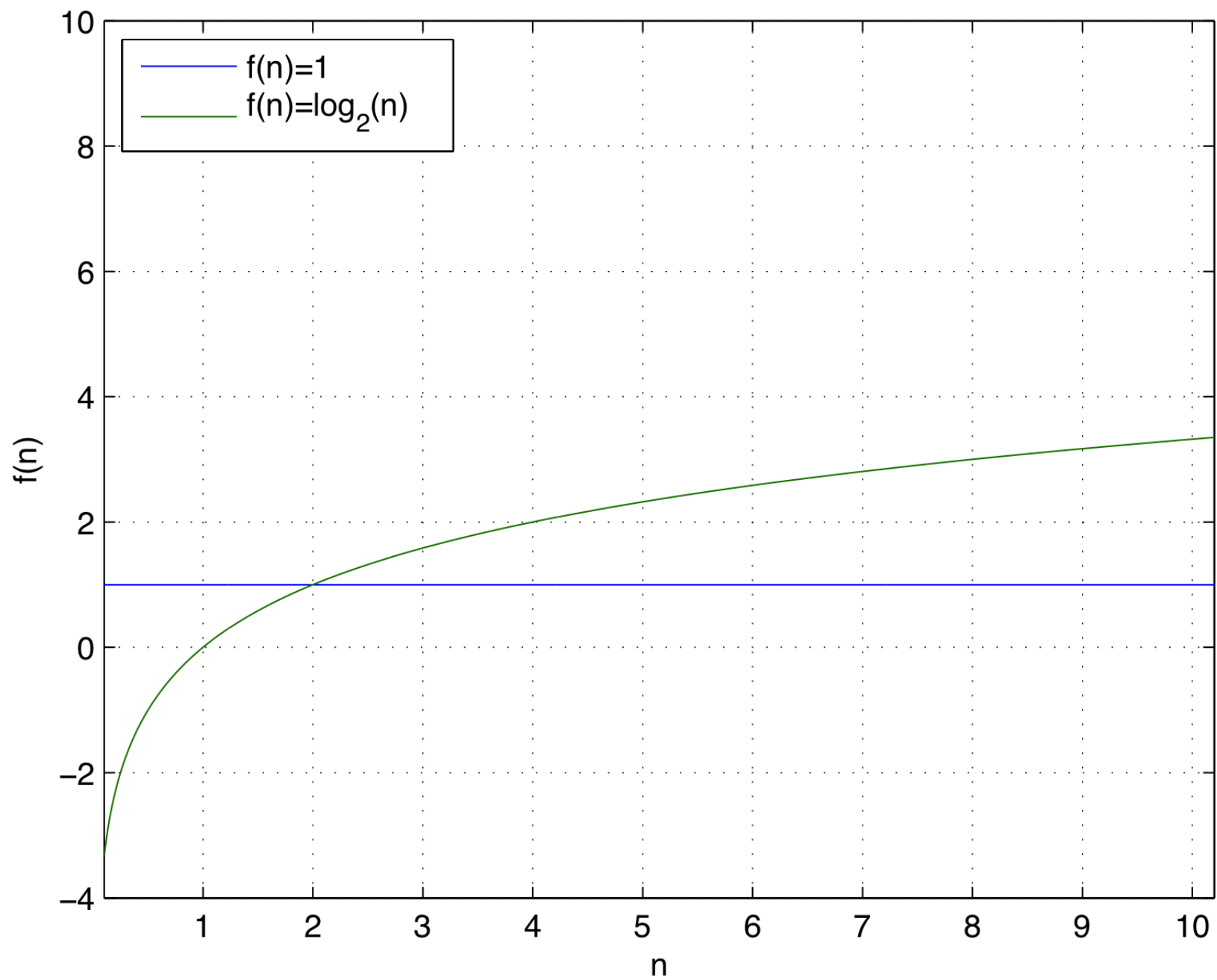
Theoretical Analysis Steps

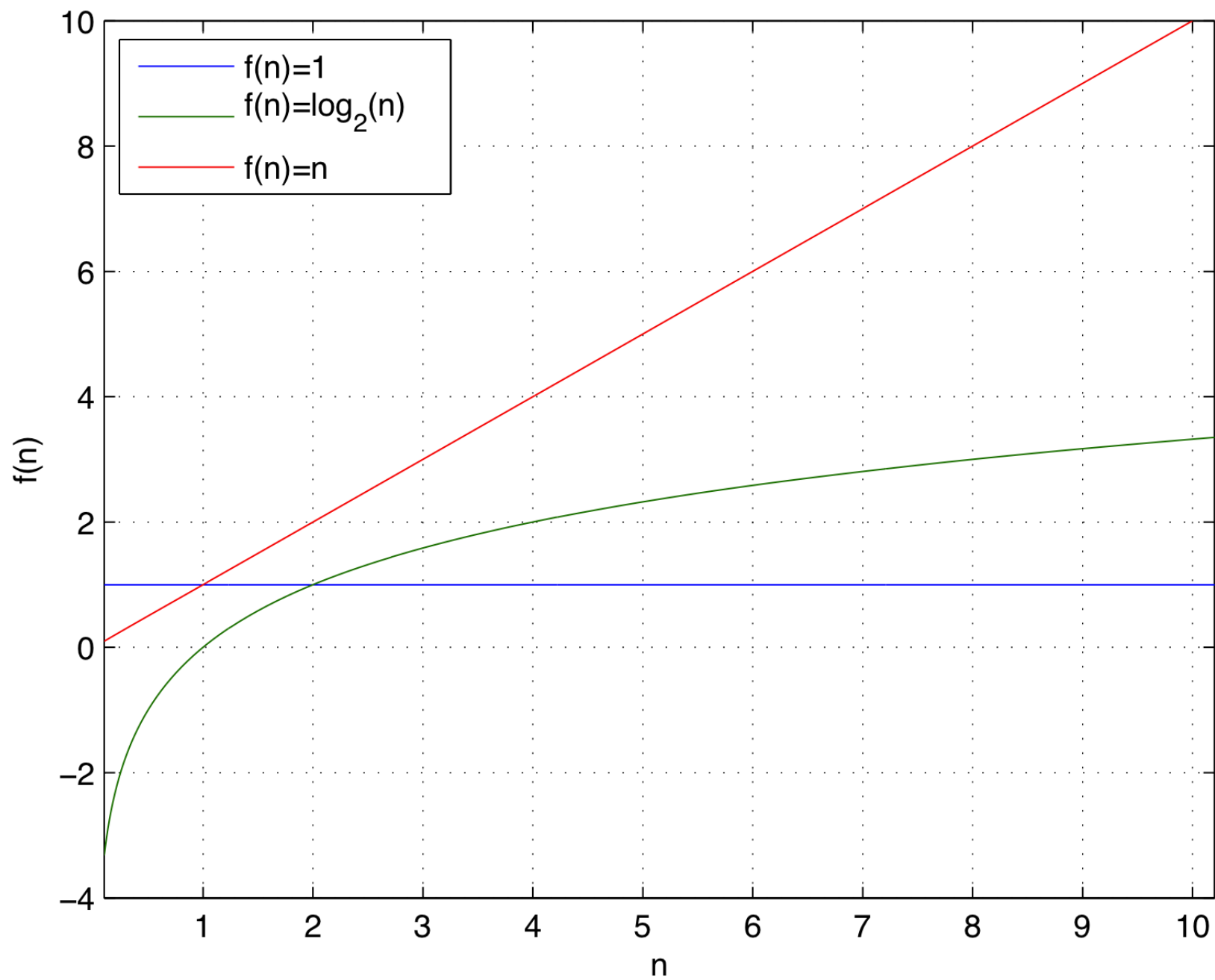
1. Express algorithm as pseudo-code
2. Count primitive operations
3. Describe algorithm as $f(n)$
 - function of n (the input size)
4. **Perform asymptotic analysis**
 - express in **asymptotic notation**

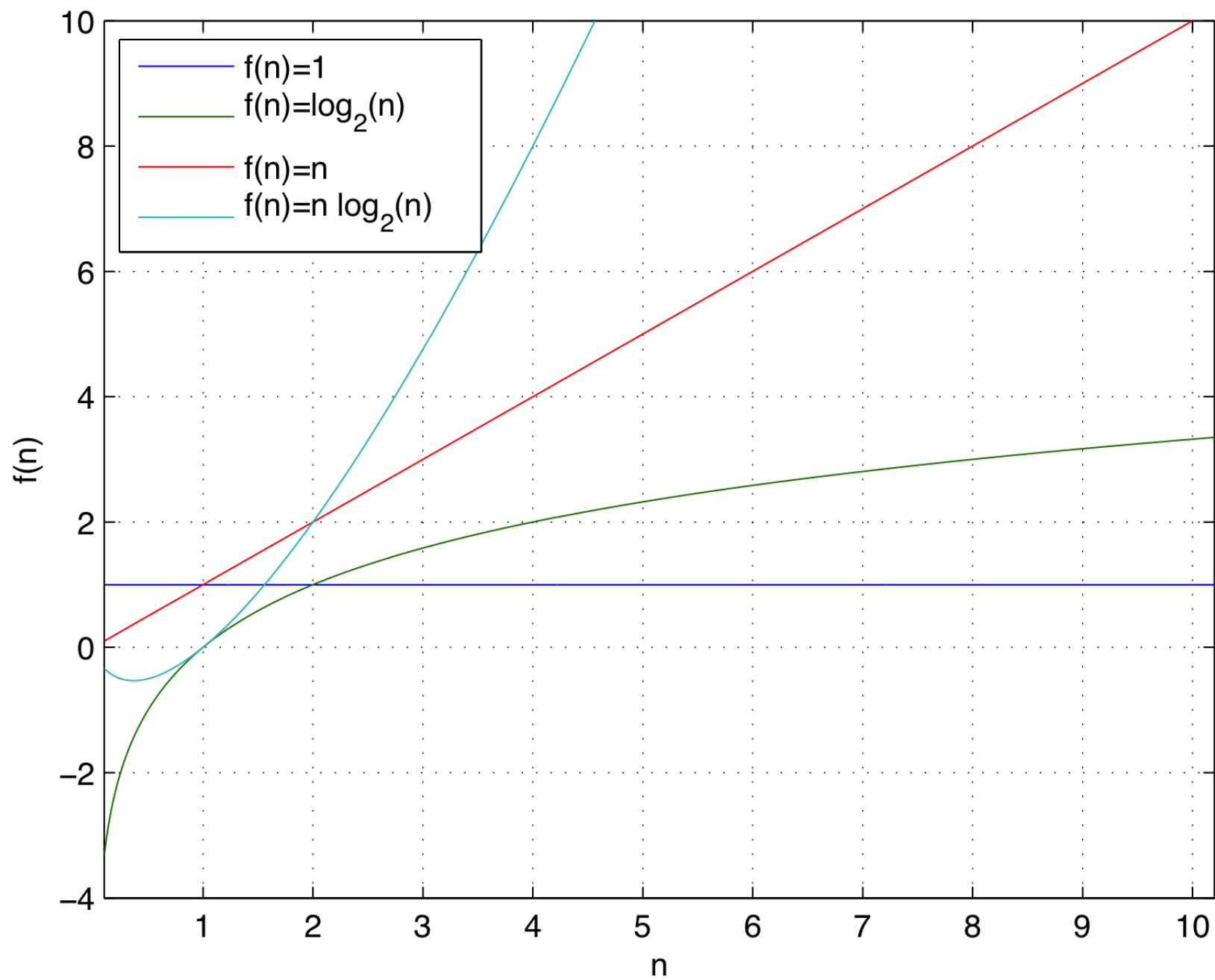
Seven Important Functions

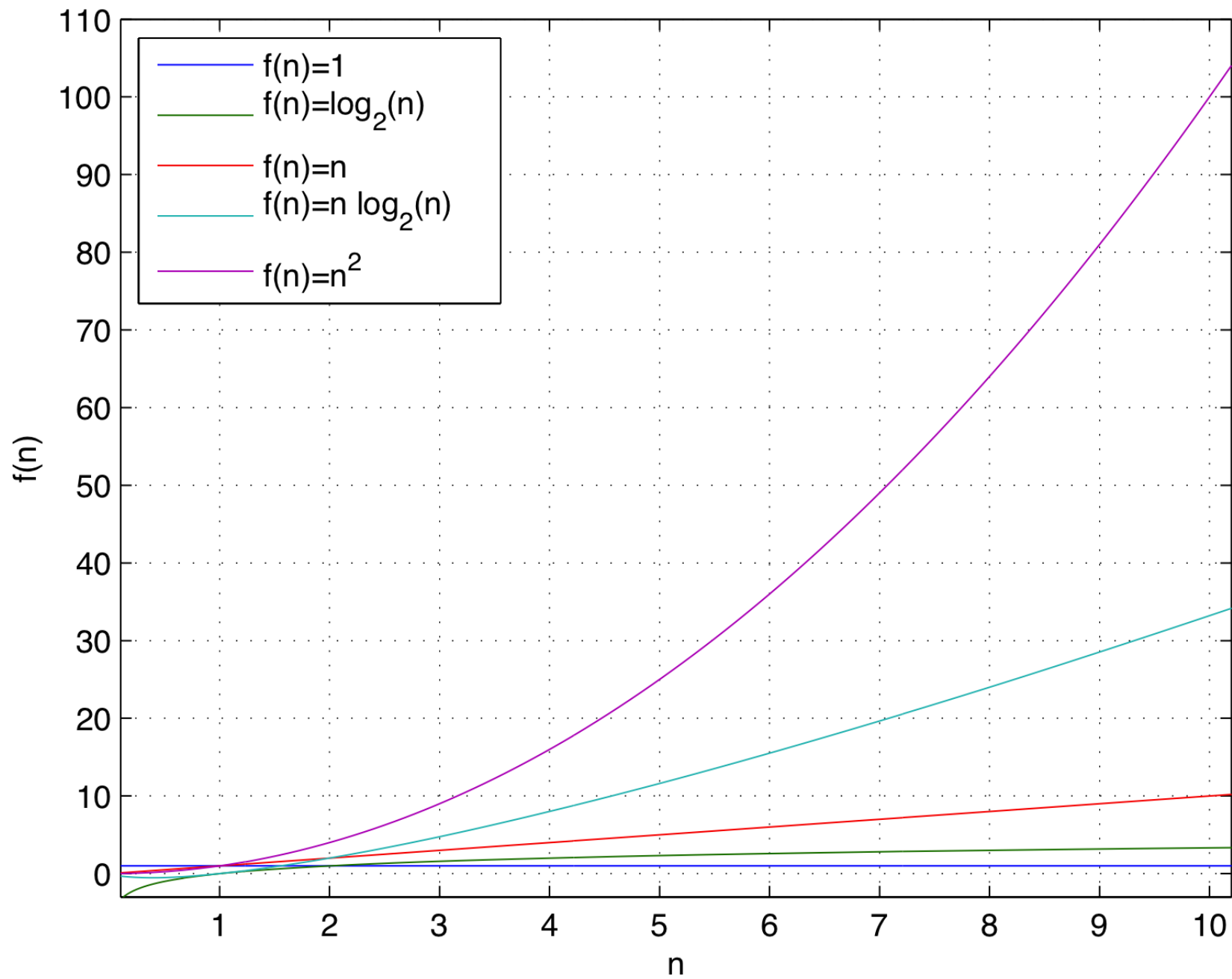
- Seven functions that often appear in algorithm analysis
 - Constant ≈ 1
 - Logarithmic $\approx \log_2 n$
 - Linear $\approx n$
 - N-Log-N $\approx n \log_2 n$
 - Quadratic $\approx n^2$
 - Cubic $\approx n^3$
 - Exponential $\approx 2^n$

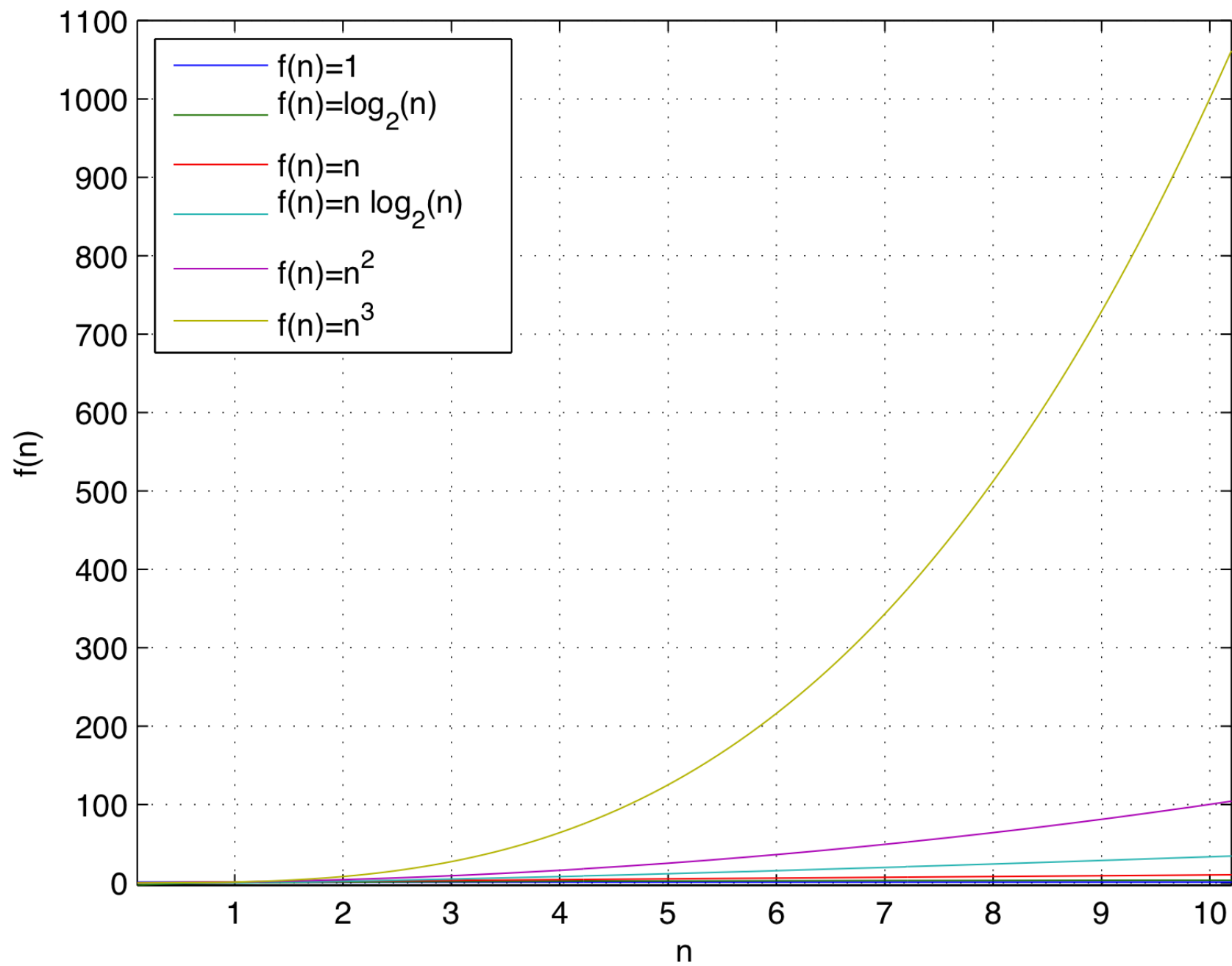


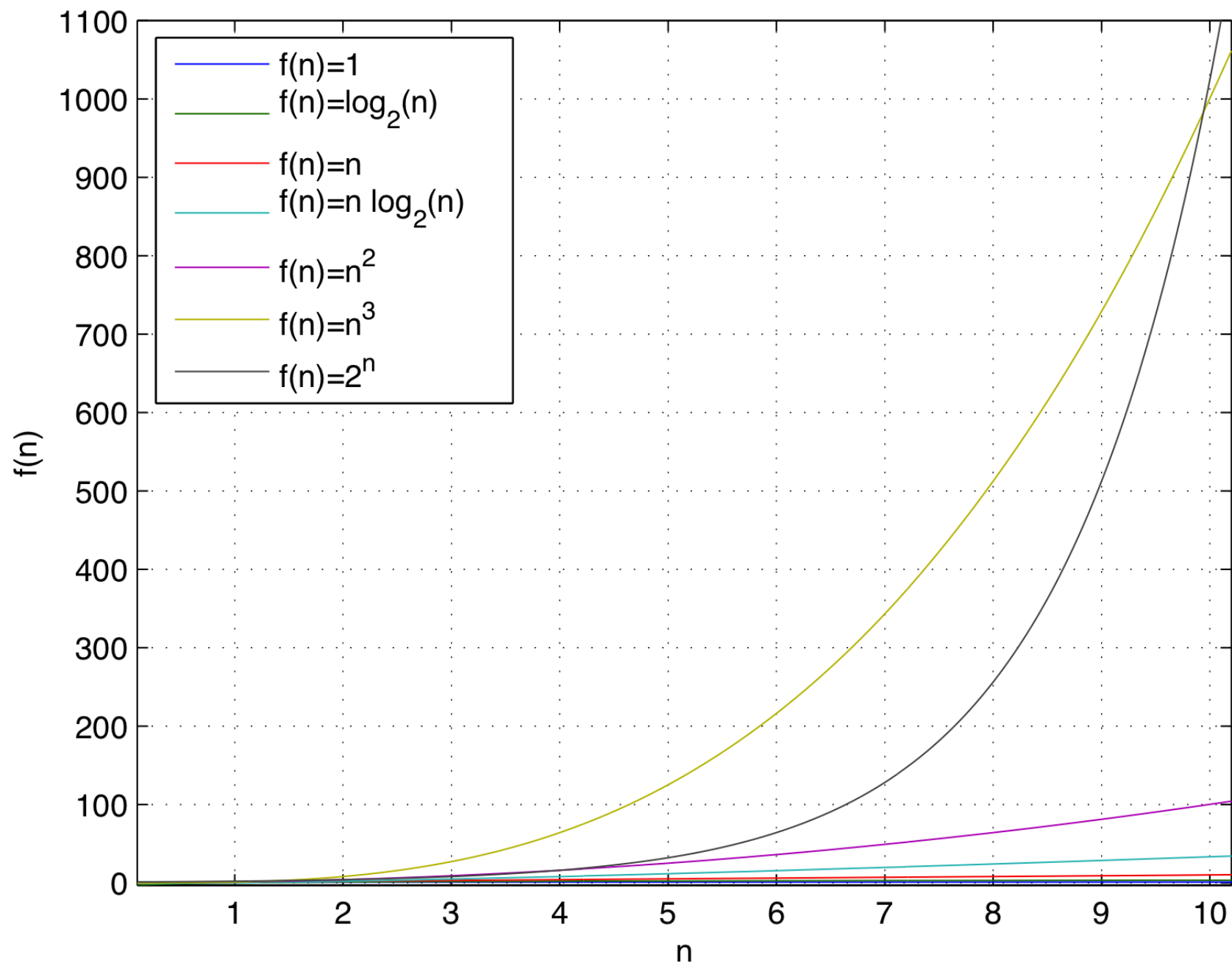








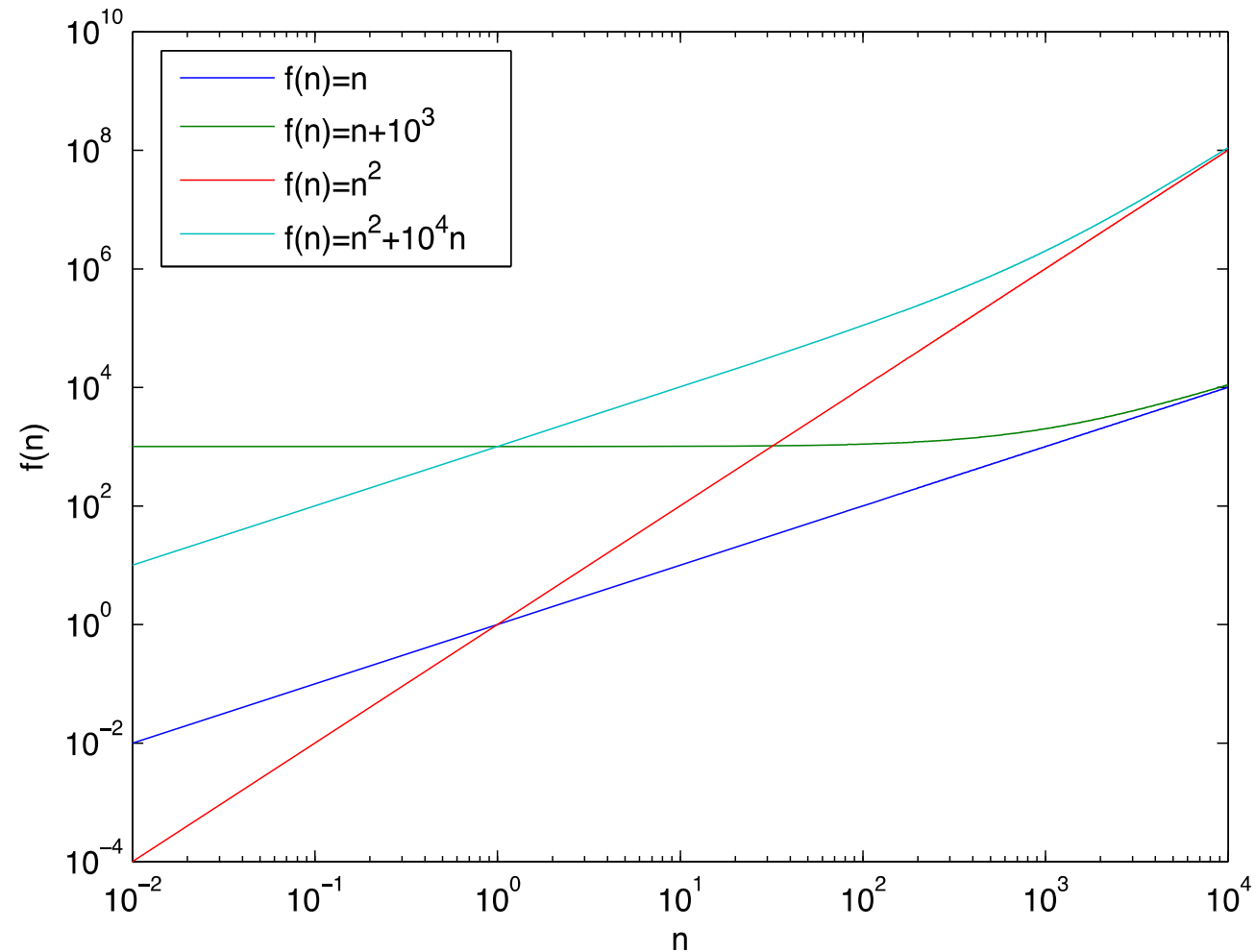




Analysis Process

- To perform an asymptotic analysis of the worst-case running time of an algorithm
 - find the worst-case number of primitive operations executed as a function of the input size – $f(n)$
 - ◆ since constant factors and lower-order terms do not affect the growth rate for large n they are usually **disregarded** when counting primitive operations
 - express this function with big-O notation

What Difference do *Lower-Order Terms* Make?



Big-O Notation

- Big-O notation describes an **upper bound** on a **function**
- $f(n)$ is $O(g(n))$ if $f(n)$ is asymptotically **less** than or equal to $g(n)$

That is,

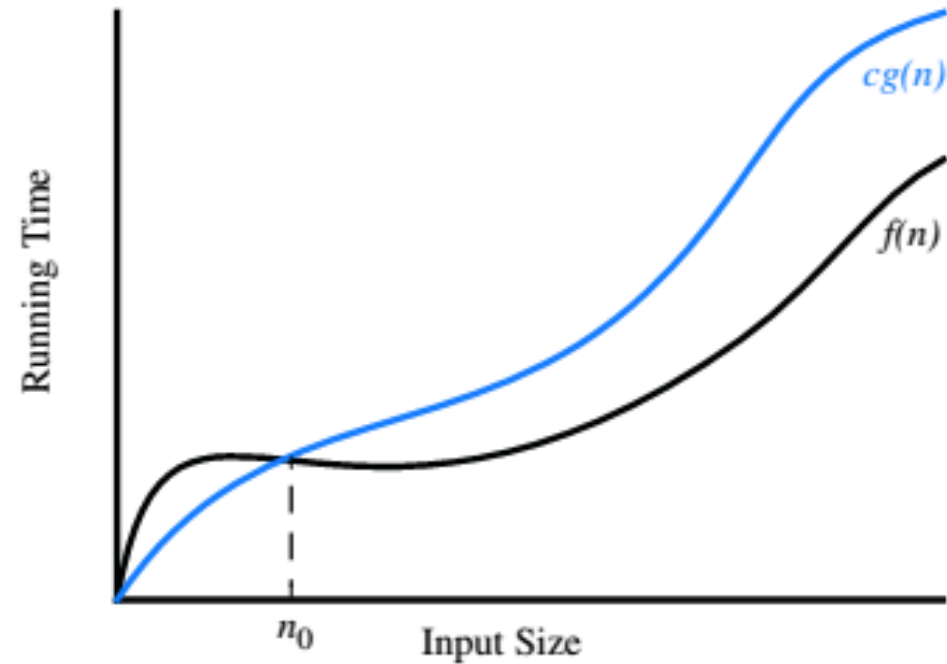
Given functions $f(n)$ and $g(n)$, we say that

$f(n)$ is $O(g(n))$

if there are positive constants c and n_0 such that

$$f(n) \leq c \cdot g(n) \text{ for } n \geq n_0$$

Illustrating Big-O Notation



- $f(n)$ is $O(g(n))$, since
- $f(n) \leq c \cdot g(n)$ when $n \geq n_0$

Big-O Example

Show that $2n + 10$ is $O(n)$

$$f(n) = 2n + 10$$

$$g(n) = n$$

$$2n + 10 \leq c \cdot n$$

$$10 \leq c \cdot n - 2n$$

$$c \cdot n - 2n \geq 10$$

$$(c - 2) n \geq 10$$

$$n \geq 10 \div (c - 2)$$

Pick $c = 3$ and $n_0 = 10$

$f(n)$ is $O(g(n))$ if there are positive constants c and n_0 such that
 $f(n) \leq c \cdot g(n)$ for $n \geq n_0$

Consider Big-O of *arrayMax*

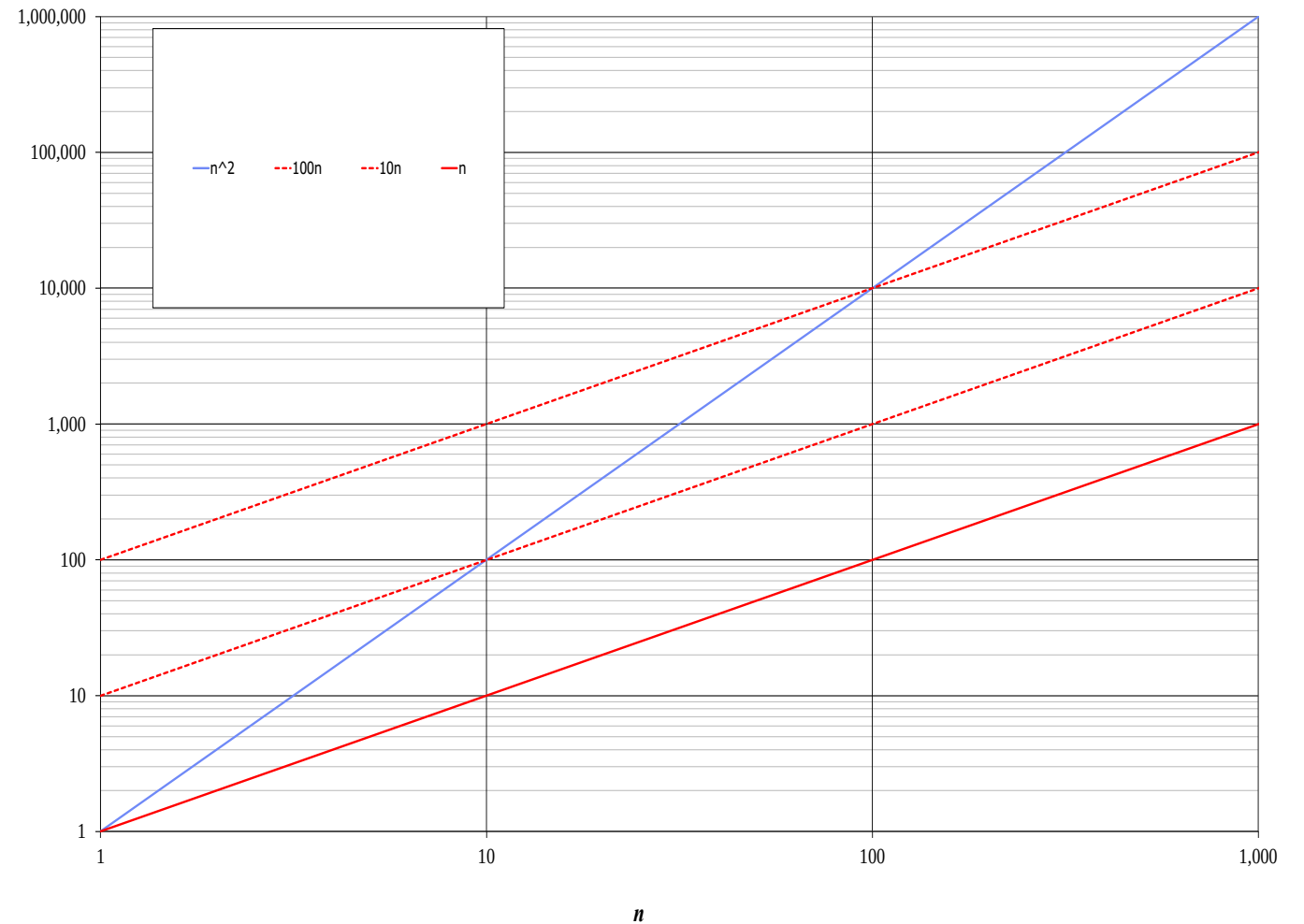
- *arrayMax* executes at most $6n$ primitive operations
 - constant factor “6” can be disregarded from $6n$
 - lower order terms can be disregarded

Thus, we say that algorithm *arrayMax*
“runs in $O(n)$ time” (or, that it runs in “linear time”)

Big-O Example

□ Example: the function n^2 is not $O(n)$

- $n^2 \leq cn$
- $n \geq n_0$
- Definition cannot be satisfied since c is a constant



Big-O and Growth Rate

- Big-O notation gives an *upper bound* on the *growth rate* of a function
- “ $f(n)$ is $O(g(n))$ ” means the growth rate of $f(n)$ is no more than the growth rate of $g(n)$
- Big-O notation ranks functions according to their growth rate

	$f(n)$ is $O(g(n))$	$g(n)$ is $O(f(n))$
$g(n)$ grows more	Yes	No
$f(n)$ grows more	No	Yes
Same growth	Yes	Yes

Big-O Rules

- **Rule 1:** If $f(n)$ is a polynomial of degree d , then $f(n)$ is $O(n^d)$
 - We can drop lower-order terms
 - We can drop constant factors (coefficients)

eg. $3n^4 + 7n^3 + 5$ is $O(n^4)$

Big-O Rules

- Can we say that " $2n$ is $O(n^2)$ " ?
 - yes – remember big-O **is just an upper bound**
 - but this doesn't give us much information about the function's growth rate
- **Rule 2:** Use the smallest possible class of functions (the "tightest" possible bound)
 - " $2n$ is $O(n)$ " instead of " $2n$ is $O(n^2)$ ", even if the latter is still mathematically correct...
 - Quiz gotcha: Is " $8n$ is $O(n^3)$ "?

Big-O Rules

- **Rule 3:** Use the simplest expression of the class
 - " $3n + 5$ is $O(n)$ " instead of " $3n + 5$ is $O(3n)$ "

More Big-O Examples

Plan for finding g in " $f(n)$ is $O(g(n))$ "

Drop lower-order terms from f

Drop constant factors from f

Tentatively:

- find "smallest" class of functions
- use "simplest" expression of class

$$7n - 2$$

$7n - 2$ is $O(n)$

need $c > 0$ and $n_0 \geq 1$ such that $7n - 2 \leq c \cdot n$ for $n \geq n_0$

true for $c = 7$ and $n_0 = 1$

$$3n^3 + 20n^2 + 5$$

$3n^3 + 20n^2 + 5$ is $O(n^3)$

need $c > 0$ and $n_0 \geq 1$ such that $3n^3 + 20n^2 + 5 \leq c \cdot n^3$ for $n \geq n_0$

true for $c = 4$ and $n_0 = 21$

$$3 \log n + 5$$

$3 \log n + 5$ is $O(\log n)$

need $c > 0$ and $n_0 \geq 1$ such that $3 \log n + 5 \leq c \cdot \log n$ for $n \geq n_0$

true for $c = 8$ and $n_0 = 2$

Asymptotic Analysis Recap

- What is the big-O running time of this algorithm?

Algorithm *insertionSort*(A, n)

for $i \leftarrow 1$ to $n - 1$ do

key $\leftarrow A[i]$
 $j \leftarrow i - 1$

while $j \geq 0$ and $A[j] > \text{key}$ do

$A[j + 1] \leftarrow A[j]$
 $j \leftarrow j - 1$
 $A[j + 1] \leftarrow \text{key}$

$O(1)$

$O(1)$

$O(n)$ worst-case
 $O(1)$ best-case

n iterations

- Worst-case (array is in descending order): $O(n^2)$
- Best-case (array is already sorted): $O(n)$

Example

□ Give an asymptotic upper bound (in big-O notation) for

- $\log 8n + \log (n^{2n})$
- $\log 8n + 2n \log n$
- $\log n + n \log n$
- **$O(n \log n)$**

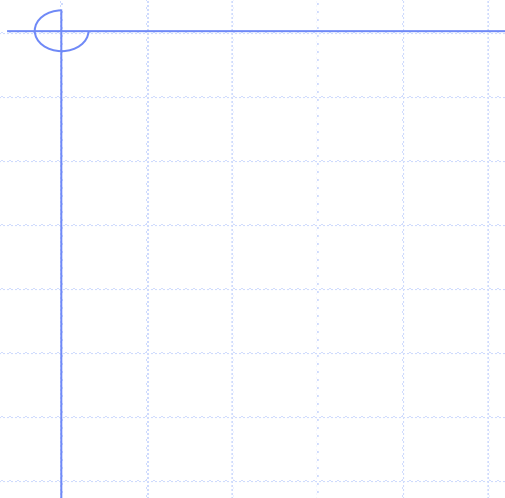
3. $\log_b a^c = c \log_b a$

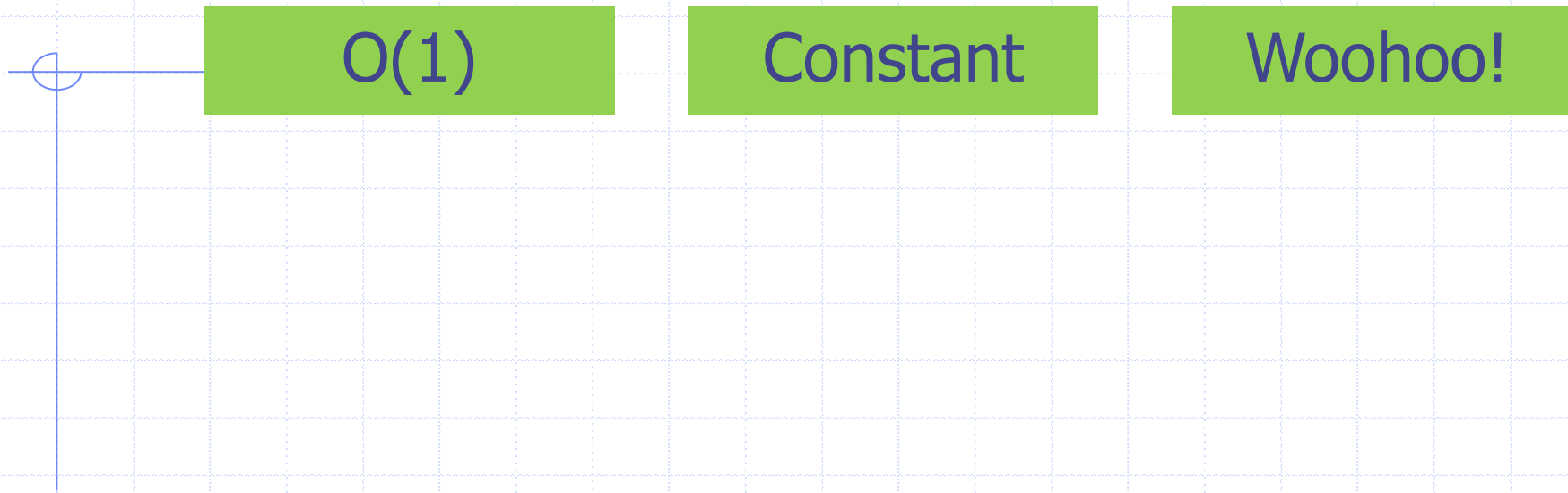
(drop constants)

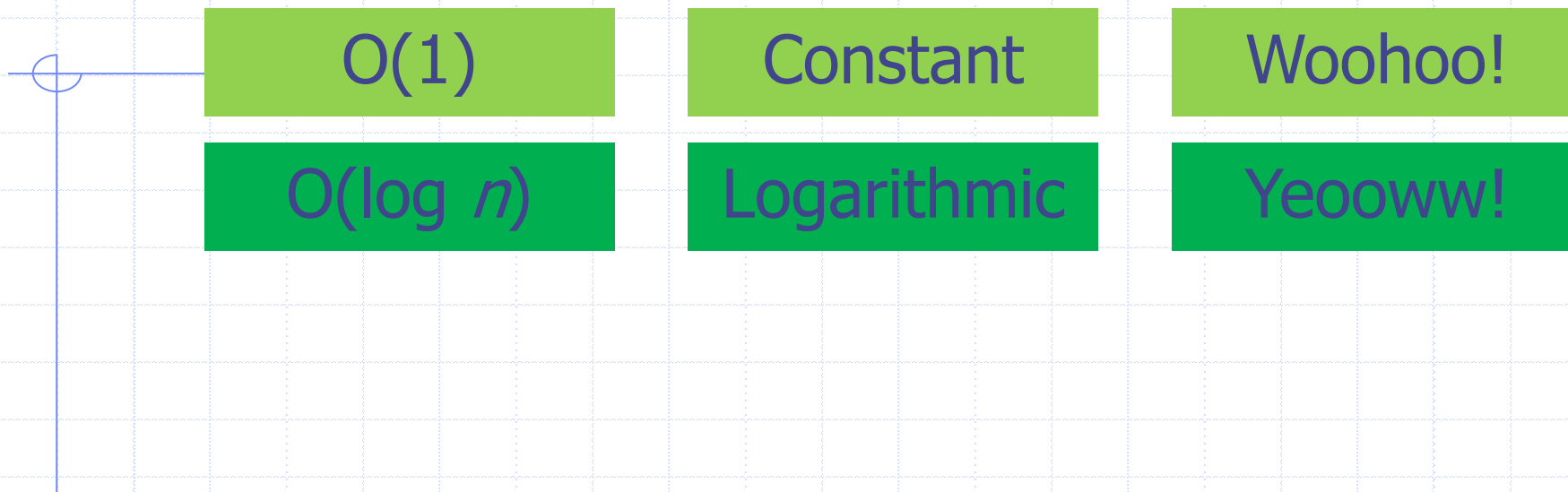
(drop lower order terms)

Let's examine these classes more closely...

Remember: We are describing the **behaviour** of the **algorithm** in terms of the size of the **input, n**







$O(1)$	Constant	Woohoo!
$O(\log n)$	Logarithmic	Yeowww!
$O(n)$	Linear	Woot!



$O(1)$	Constant	Woohoo!
$O(\log n)$	Logarithmic	Yeooww!
$O(n)$	Linear	Woot!
$O(n \log n)$	Super linear	Noice!



$O(1)$	Constant	Woohoo!
$O(\log n)$	Logarithmic	Yeooww!
$O(n)$	Linear	Woot!
$O(n \log n)$	Super linear	Noice!
$O(n^2)$	Quadratic	OK, fair!



$O(1)$	Constant	Woohoo!
$O(\log n)$	Logarithmic	Yeooww!
$O(n)$	Linear	Woot!
$O(n \log n)$	Super linear	Noice!
$O(n^2)$	Quadratic	OK, fair!
$O(n^3)$	Cubic	Tolerable



$O(1)$	Constant	Woohoo!
$O(\log n)$	Logarithmic	Yeooww!
$O(n)$	Linear	Woot!
$O(n \log n)$	Super linear	Noice!
$O(n^2)$	Quadratic	OK, fair!
$O(n^3)$	Cubic	Tolerable
$O(2^n)$	Exponential	Intractable

If I was given 10^{10} loop iterations
= 10000000000

(10 billion – Maybe 10-30 seconds worth of computation,
depends on the language/compiler/computer/...)

What value of n could we reach?



$O(1)$	Constant	Woohoo!
$O(\log n)$	Logarithmic	Yeooww!
$O(n)$	Linear	Woot!
$O(n \log n)$	Super linear	Noice!
$O(n^2)$	Quadratic	OK, fair!
$O(n^3)$	Cubic	Tolerable
$O(2^n)$	Exponential	Intractable



$O(1)$	Constant	Woohoo!	
$O(\log n)$	Logarithmic	Yeooww!	
$O(n)$	Linear	Woot!	
$O(n \log n)$	Super linear	Noice!	
$O(n^2)$	Quadratic	OK, fair!	
$O(n^3)$	Cubic	Tolerable	
$O(2^n)$	Exponential	Intractable	Tiny, 33



$O(1)$	Constant	Woohoo!	
$O(\log n)$	Logarithmic	Yeooww!	
$O(n)$	Linear	Woot!	
$O(n \log n)$	Super linear	Noice!	
$O(n^2)$	Quadratic	OK, fair!	
$O(n^3)$	Cubic	Tolerable	Modest, 2150
$O(2^n)$	Exponential	Intractable	Tiny, 33



$O(1)$	Constant	Woohoo!	
$O(\log n)$	Logarithmic	Yeooww!	
$O(n)$	Linear	Woot!	
$O(n \log n)$	Super linear	Noice!	
$O(n^2)$	Quadratic	OK, fair!	Biggish, 10^5
$O(n^3)$	Cubic	Tolerable	Modest, 2^{150}
$O(2^n)$	Exponential	Intractable	Tiny, 33



$O(1)$	Constant	Woohoo!	
$O(\log n)$	Logarithmic	Yeooww!	
$O(n)$	Linear	Woot!	
$O(n \log n)$	Super linear	Noice!	Large, $10^{8.5}$
$O(n^2)$	Quadratic	OK, fair!	Biggish, 10^5
$O(n^3)$	Cubic	Tolerable	Modest, 2150
$O(2^n)$	Exponential	Intractable	Tiny, 33



$O(1)$	Constant	Woohoo!	
$O(\log n)$	Logarithmic	Yeooww!	
$O(n)$	Linear	Woot!	Huge, 10^{10}
$O(n \log n)$	Super linear	Noice!	Large, $10^{8.5}$
$O(n^2)$	Quadratic	OK, fair!	Biggish, 10^5
$O(n^3)$	Cubic	Tolerable	Modest, 2150
$O(2^n)$	Exponential	Intractable	Tiny, 33



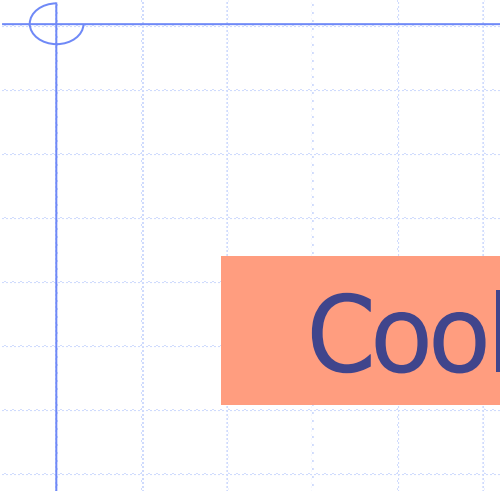
$O(1)$	Constant	Woohoo!	
$O(\log n)$	Logarithmic	Yeooww!	Astronomical
$O(n)$	Linear	Woot!	Huge, 10^{10}
$O(n \log n)$	Super linear	Noice!	Large, $10^{8.5}$
$O(n^2)$	Quadratic	OK, fair!	Biggish, 10^5
$O(n^3)$	Cubic	Tolerable	Modest, 2^{150}
$O(2^n)$	Exponential	Intractable	Tiny, 33



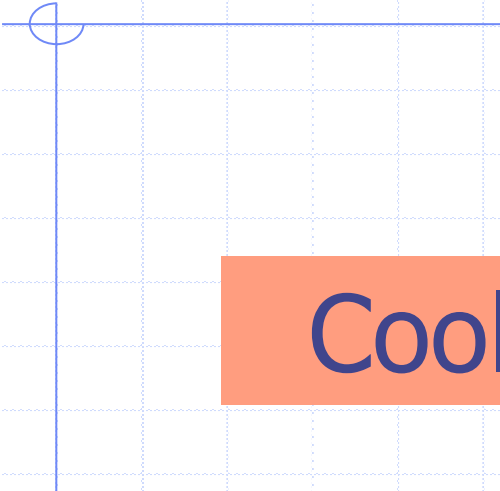
$O(1)$	Constant	Woohoo!	To infinity
$O(\log n)$	Logarithmic	Yeooww!	Astronomical
$O(n)$	Linear	Woot!	Huge, 10^{10}
$O(n \log n)$	Super linear	Noice!	Large, $10^{8.5}$
$O(n^2)$	Quadratic	OK, fair!	Biggish, 10^5
$O(n^3)$	Cubic	Tolerable	Modest, 2150
$O(2^n)$	Exponential	Intractable	Tiny, 33



$O(1)$	Constant	Woohoo!	AND BEYOND!!
$O(\log n)$	Logarithmic	Yeooww!	Astronomical
$O(n)$	Linear	Woot!	Huge, 10^{10}
$O(n \log n)$	Super linear	Noice!	Large, $10^{8.5}$
$O(n^2)$	Quadratic	OK, fair!	Biggish, 10^5
$O(n^3)$	Cubic	Tolerable	Modest, 2150
$O(2^n)$	Exponential	Intractable	Tiny, 33

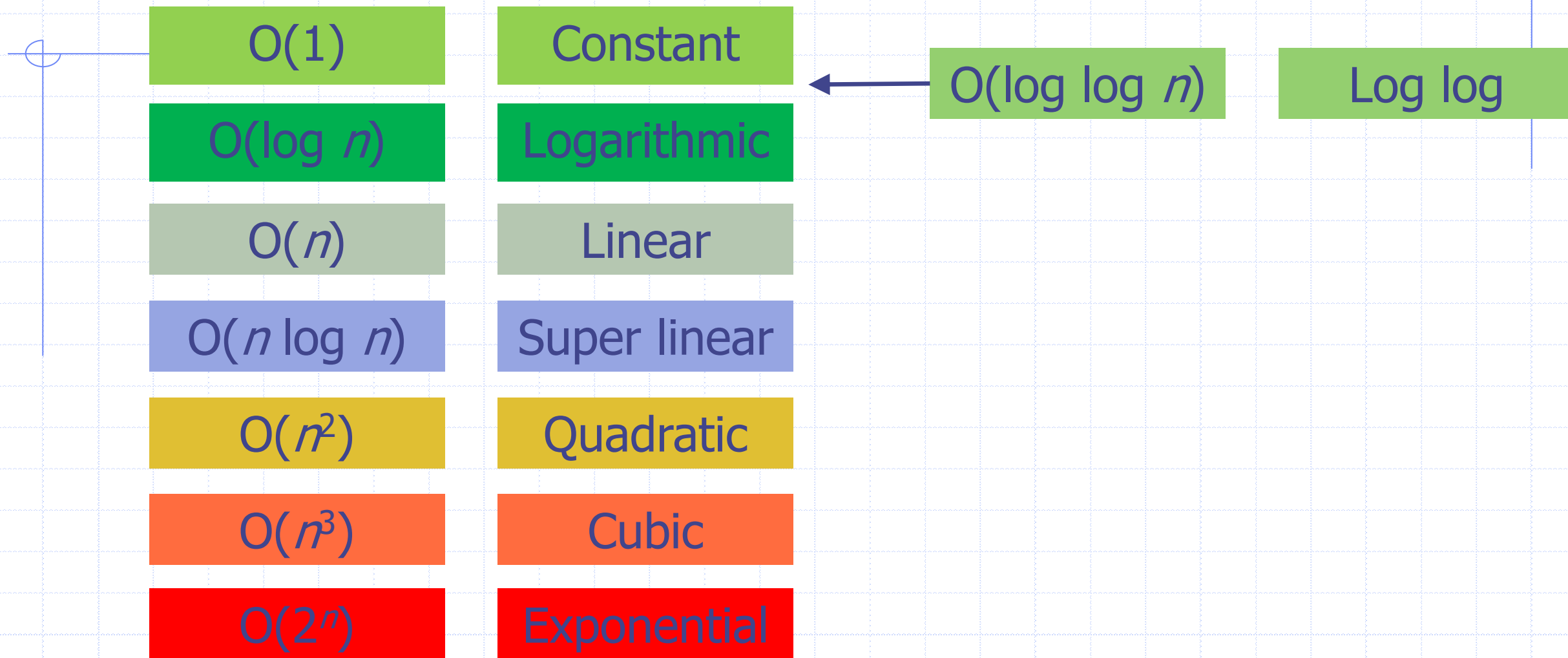


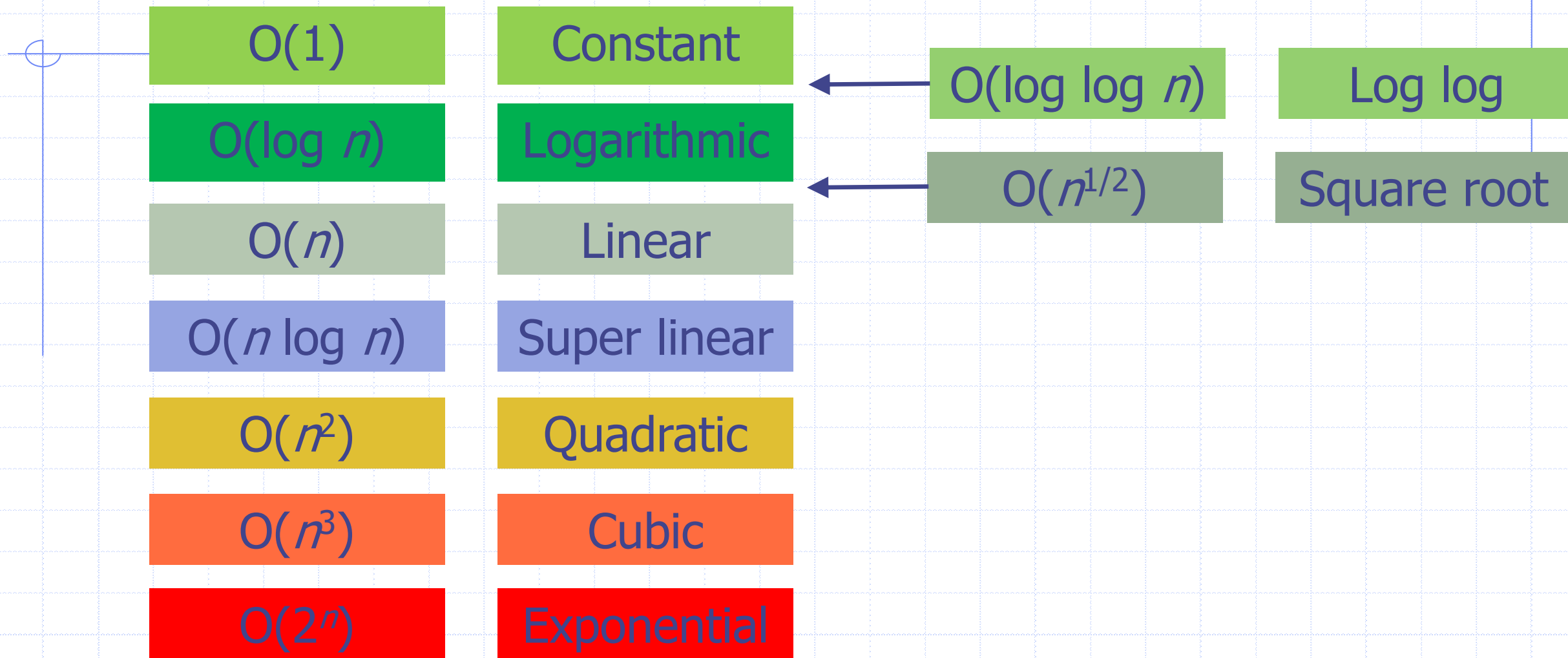
Cool! But what about other functions?

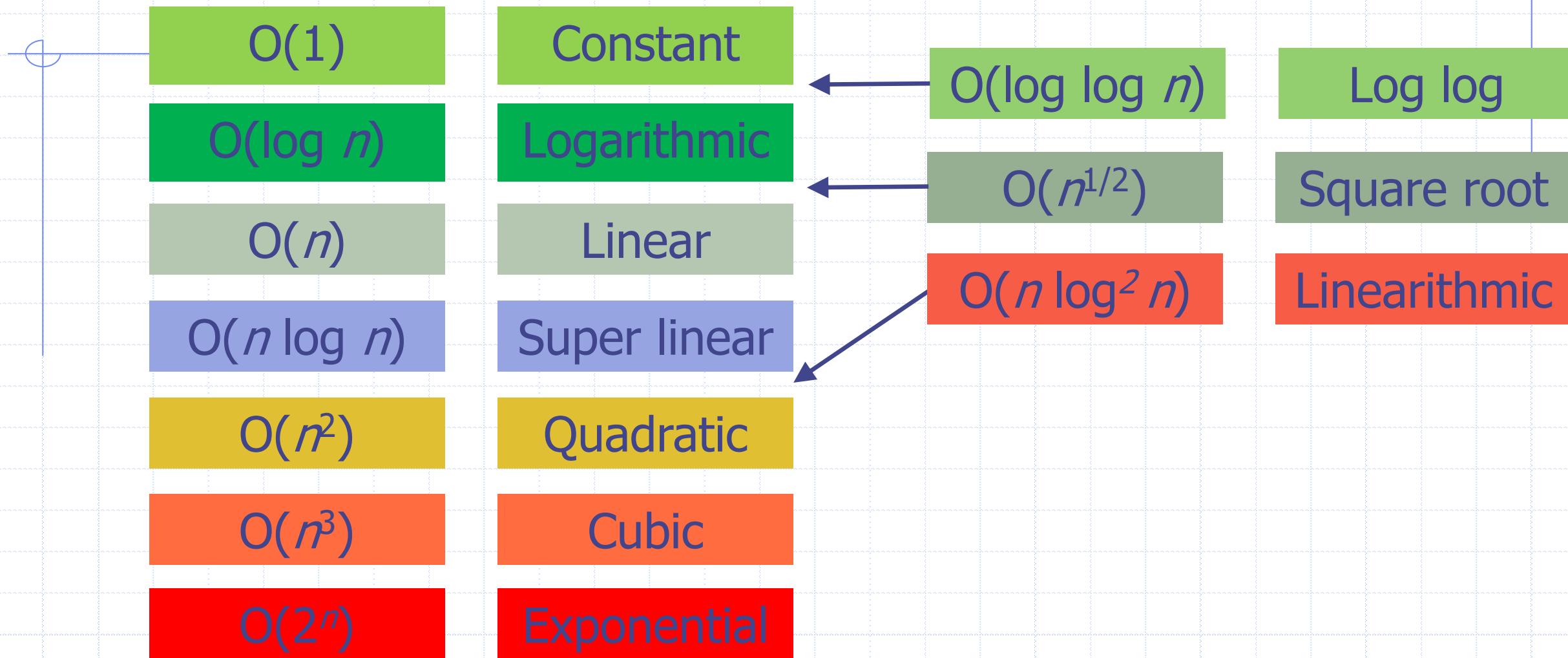


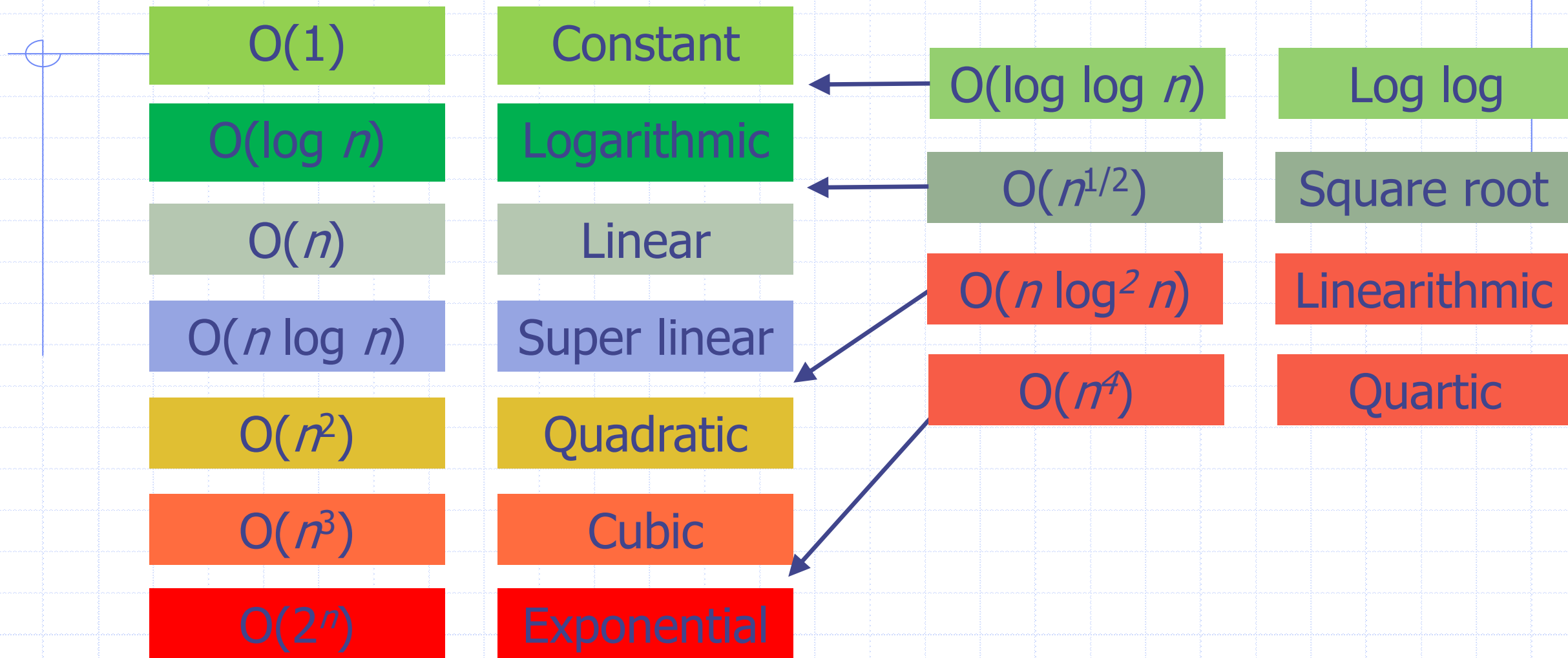
Cool! But what about other functions?

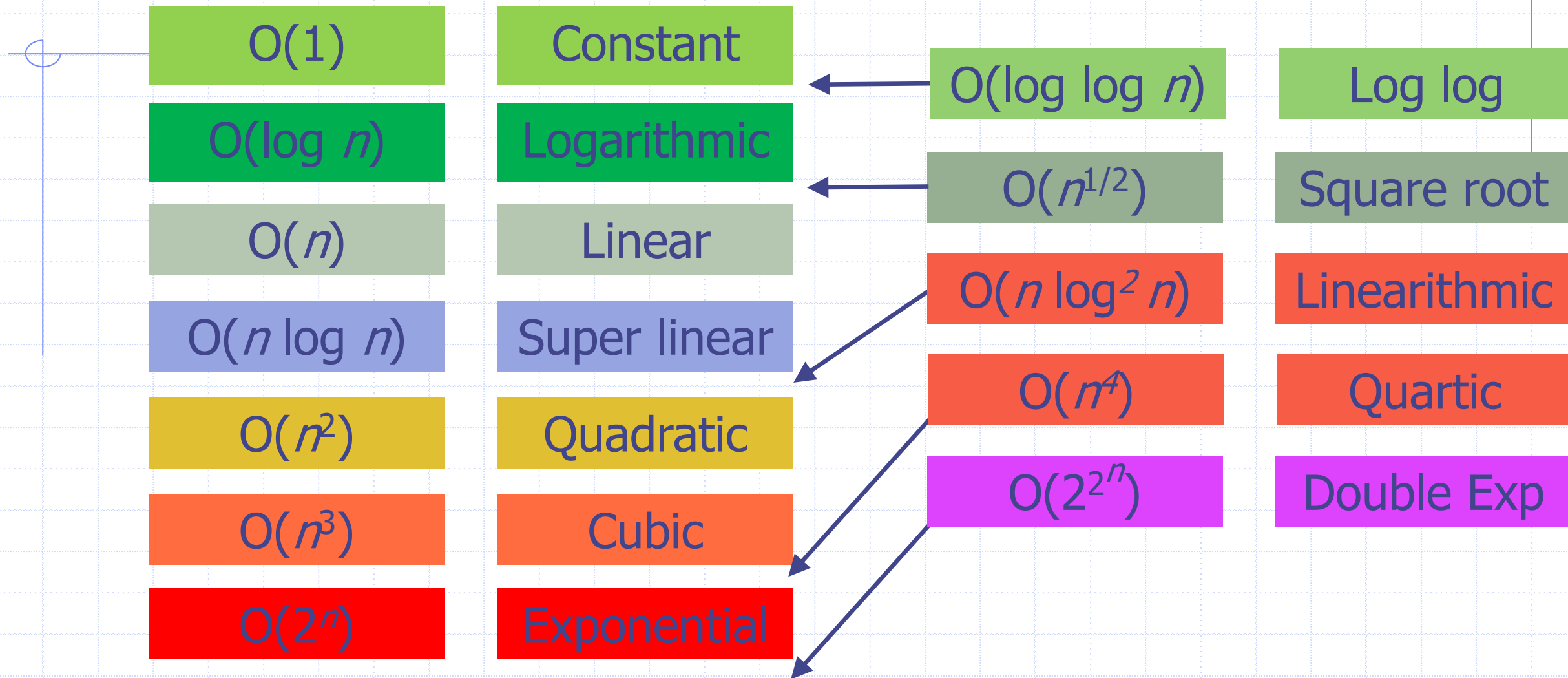
We can use as many as we want!



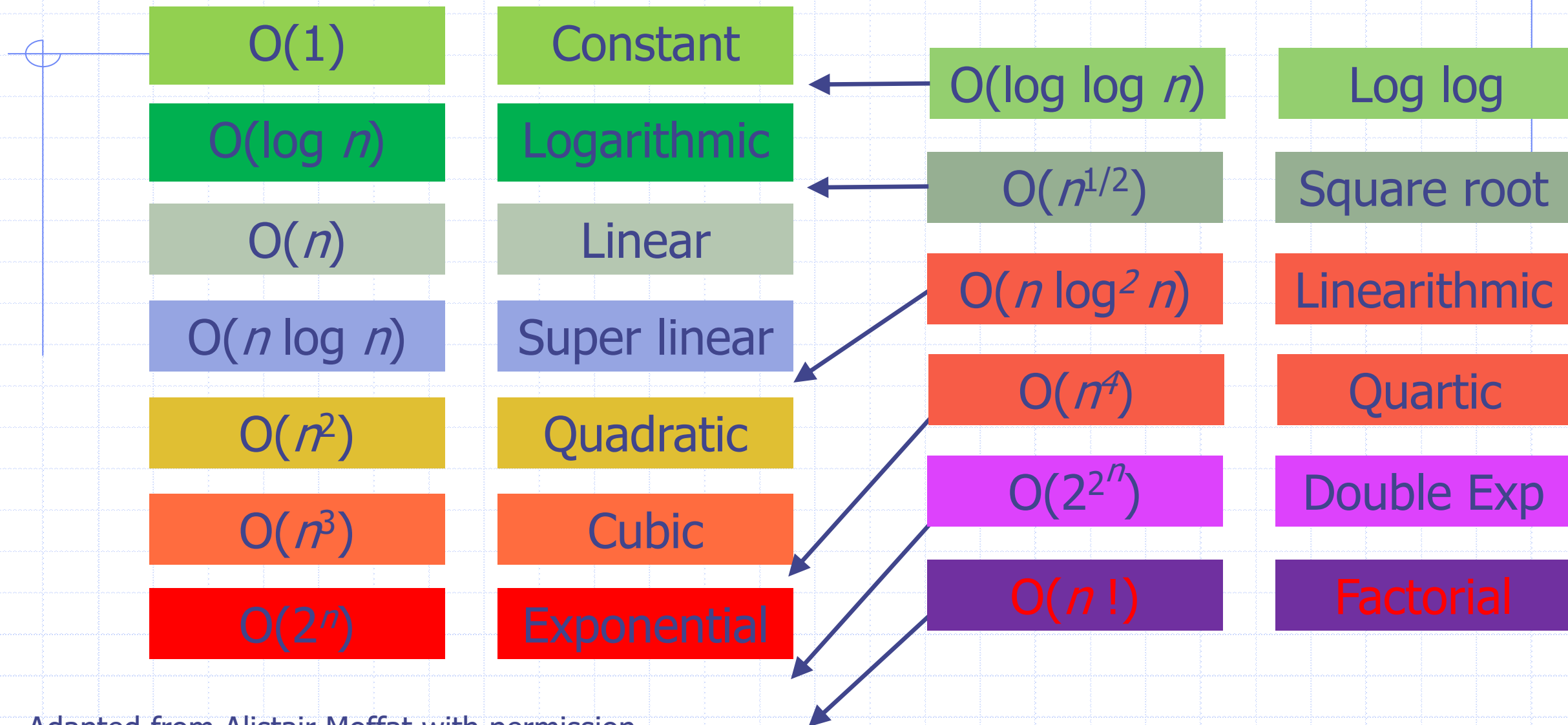








Adapted from Alistair Moffat with permission



Adapted from Alistair Moffat with permission

Big-Omega Notation

- Big-Omega notation describes a lower bound on a function
- $f(n)$ is $\Omega(g(n))$ if $f(n)$ is asymptotically **greater** than or equal to $g(n)$
- That is,

$f(n)$ is $\Omega(g(n))$ if there exist positive constants c and n_0 such that
 $f(n) \geq c \cdot g(n)$ for all $n \geq n_0$

Big-O vs Big-Omega

Given functions $f(n)$ and $g(n)$, we say that:

$f(n)$ is $O(g(n))$ if there are positive constants c and n_0 such that

$$f(n) \leq c \cdot g(n) \text{ for all } n \geq n_0$$

$f(n)$ is $\Omega(g(n))$ if there exist positive constants c and n_0 such that

$$f(n) \geq c \cdot g(n) \text{ for all } n \geq n_0$$

Big-Theta Notation

- Big-Theta notation describes a **tight bound** on a function (if one exists)
- $f(n)$ is $\Theta(g(n))$ if $f(n)$ is asymptotically **equal** to $g(n)$
- That is,

$f(n)$ is $\Theta(g(n))$ if there exist positive constants c_1 , c_2 , and n_0 such that $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ for all $n \geq n_0$

- By definition, if $f(n)$ is in $O(g(n))$ **and** $f(n)$ is in $\Omega(g(n))$, then $f(n)$ must be $\Theta(g(n))$

Using Relatives of Big-O

$5n^2$ is $\Omega(n^2)$

let $c = 1$ and $n_0 = 1$

$f(n)$ is $\Omega(g(n))$ if there is a constant $c > 0$ and an integer constant $n_0 \geq 1$ such that $f(n) \geq c \cdot g(n)$ for $n \geq n_0$

$5n^2$ is $O(n^2)$

let $c = 5$ and $n_0 = 1$

$f(n)$ is $O(g(n))$ if there is a constant $c > 0$ and an integer constant $n_0 \geq 1$ such that $f(n) \leq c \cdot g(n)$ for $n \geq n_0$

$5n^2$ is $\Theta(n^2)$

$f(n)$ is $\Theta(g(n))$ if it is $\Omega(n^2)$ and $O(n^2)$.

Let $c = 5$ and $n_0 = 1$

$c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ for all $n \geq n_0$

$5 \cdot n^2 \leq 5n^2 \leq 5 \cdot n^2$ for all $n \geq 1$



Final word: How scary is **exponential** growth?

Suppose I asked to be paid 1 billionth of a cent for the first student that passed this course...

Suppose I asked to be paid 1 billionth of a cent for the first student that passed this course...

And 2 billionths of a cent for the second...

Suppose I asked to be paid 1 billionth of a cent for the first student that passed this course...

And 2 billionths of a cent for the second...

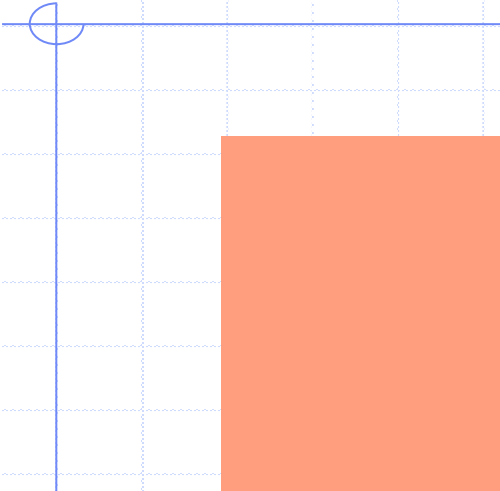
And 4 billionths of a cent for the third...

Suppose I asked to be paid 1 billionth of a cent for the first student that passed this course...

And 2 billionths of a cent for the second...

And 4 billionths of a cent for the third...

Would you accept this contract?



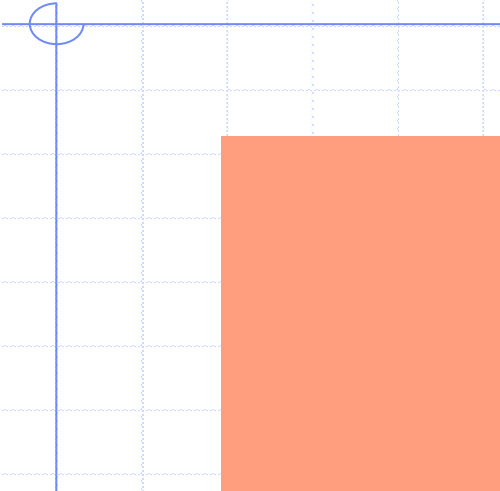
Suppose I pass 30 of you 😊

That's 2^{30} billionths of a cent

Suppose I pass 30 of you 😊

That's 2^{30} billionths of a cent

Or about 1-2 cents 😞



Suppose I pass 40 of you 😊

That's 2^{40} billionths of a cent

Suppose I pass 40 of you 😊

That's 2^{40} billionths of a cent

Or about \$11 😞

Suppose I pass 100 of you 😊

That's 2^{100} billionths of a cent

Suppose I pass 100 of you 😊

That's 2^{100} billionths of a cent

Or about \$12,676,506,002,282,294,014

Suppose I pass everyone...

That's 2^{600} billionths of a cent

$\$4.15 \times 10^{169}$

Further Reading

- Data Structures and Algorithms in Python
 - Chapter 3.1 (the rest of 3 is also interesting and useful)
 - Chapter 4
- Introduction to Algorithms
 - Chapter 2.1, 2.2